

Привет, у меня есть идея, хочу сделать тг бота тур агента, у него будет две ветки, подобрать с нуля и там спрашиваем информацию кратко и потом после получения какой-то информации мы идем в интернет и консультируем по выбранному направлению и отвечаем на вопросы, а в ветке подобрать с нуля надо чтобы собирали всю информацию и потом шли в гугл таблицу, брали тур из нужного направления там их несколько и выводил пользователю, в финале анкета берем данные и передаем менеджеру, пока реально менеджеру не передаем, но после получения номера и Имя Фамилия мы выводим сообщение что передали ваше заявку ожидайте

Давай продумаем нужные кнопки инлайн, реплай и соберем финально что это и как будет выглядеть чтобы на основании этого тз собрать тг бота

Идея изначально такая

Думал на протяжении 14s >

Ниже — "скелет" ТЗ: как будет выглядеть бот для тур-агента с **2 ветками**, какие **reply/inline кнопки** нужны и как идет логика от старта до "заявка передана".

1) Главный экран и навигация

Reply-кнопки (постоянные, внизу клавиатуры)

1.  Подобрать тур с нуля
2.  Консультация по направлению
3.  Моя заявка / анкета (если пользователь уже дошёл до финала)
4.  Меню
5.  Назад

Почему так: reply — это "режимы" и навигация. Они **статичные**, не прыгают.

Inline-кнопки (внутри сообщений, контекстные)

- 📝 Изменить параметры (в анкете/после подбора)
 - 🎯 Показать туры (когда собрали фильтры)
 - 🔄 Показать ещё (пагинация туров)
 - ✅ Выбрать этот тур (на карточке тура)
 - 📌 Сохранить в избранное (опционально)
 - ❓ Задать вопрос (в режиме консультации)
 - 🎧 Озвучить (если хочешь фичу озвучки; на карточках/ответах)
 - 🏠 В меню
-

2) Ветка А: "Консультация по направлению" (быстро → интернет → ответы)

Цель

Пользователь **уже примерно знает направление** (или выбирает из списка), бот:

1. уточняет базу (даты/бюджет/кто едет — очень кратко),
2. переходит в режим консультации: отвечает на вопросы, даёт советы, нюансы, "что посмотреть", сезонность, визы/прививки и т.д.
3. при желании — предлагает перейти к подбору туров из таблицы или оставить заявку.

Сообщение старта (пример)

"Ок! По какому направлению консультация?

Выбери из списка или напиши своё."

Inline варианты направлений (если у тебя топ-направления):

Непал / Сафари / Абхазия / Байкал / Круиз / ... + "Другое"

Мини-уточнение (3 вопроса максимум)

1. Даты или месяц поездки?
2. Сколько дней?
3. Бюджет "примерно" (на человека/на семью)?

После этого бот пишет:

"Принял. Задавай вопросы по направлению — отвечу и помогу спланировать."

Inline в этом режиме:

- ❓ Частые вопросы (покажет меню FAQ)

-  Перейти к подбору туров
-  Оставить заявку

3) Ветка В: "Подобрать тур с нуля" (анкета → Google Sheets → выдача туров)

Цель

Собираем полные параметры → идём в Google Sheets → вытаскиваем несколько туров подходящего направления → показываем карточки → пользователь выбирает → финальная анкета → "передали менеджеру".

Этап В1: Быстрая анкета (8–10 пунктов, без воды)

Рекомендую такой порядок (коротко и по делу):

1. Куда хотите?
 - Inline: список направлений + "Не знаю (помоги выбрать)"
2. Город вылета
3. Даты (или диапазон)
4. Длительность (ночей/дней)
5. Состав: взрослые / дети (возраст детей)
6. Бюджет (общий или на человека)
7. Тип отдыха
 - Inline:  пляж /  горы /  сафари /  спокойный /  активный /  семейный /  романтика
8. Комфорт (эконом/средний/премиум)
9. Питание (без разницы / завтраки / всё включено)
10. Важные пожелания (1 сообщением)

Reply в анкете:  Назад,  Меню

Inline:  Изменить (показывает список полей для правки),  Всё верно → "Показать туры"

Этап В2: Поиск в Google Sheets

После " Всё верно" бот:

- фильтрует по **направлению** (и по желанию: сезон/даты/бюджет/тип),
- выбирает **3–5 лучших** (или 3 сразу + "Показать ещё").

Как должна выглядеть строка тура в Google Sheets (рекомендую колонки)

- direction (Непал/Сафари/...)
- tour_title
- short_desc (1–2 строки)
- duration_days
- price_from
- season (когда лучше)
- highlights (через ";")
- included (что включено)
- image_url (прямая ссылка/drive link)
- booking_link (если есть)
- tags (семья/актив/комфорт...)

Тогда бот легко соберёт "карточку тура" без фантазий.

Этап В3: Карточка тура (шаблон сообщения)

Название тура

Коротко: ...

Длительность: ...

Цена от: ...

Ключевые фишки: ...

Фото: (ссылка)

Inline на карточке:

-  Выбрать этот тур
-  Следующий / Показать ещё
-  Вопрос по туру
-  Озвучить (если нужно)

4) Финал: анкета + "передали менеджеру"

Когда пользователь нажал " Выбрать этот тур":

1. Имя
2. Фамилия
3. Телефон
4. Удобное время для связи (утро/день/вечер или интервал)

5. Комментарий (опционально)

После этого:

"Спасибо! Заявку передали менеджеру. Ожидайте, с вами свяжутся в указанное время."

Inline:

- 🏠 В меню
- 📝 Изменить заявку

5) Поведение "чтобы бот выглядел умным и не бесил"

- Всегда показывать прогресс анкеты: "Шаг 3/8"
- Одно сообщение = один вопрос (кроме шага "важные пожелания")
- Не придумывать туры, если из таблицы ничего не найдено:
"По этим условиям туров нет. Давай ослабим фильтры: бюджет / даты / направление" + inline варианты.
- Пагинация: показывать 3 тура, потом "Показать ещё".

6) Мини-пример сценария (как выглядит для пользователя)

Пользователь: 🛒 Подобрать тур с нуля

Бот: "Шаг 1/8. Куда хотите поехать?" + (inline направления + "Не знаю")

...

Бот: "Шаг 8/8. Напишите важные пожелания одним сообщением."

Пользователь: "Хочу без экстрима, комфорт, красивые виды."

Бот: "Супер. Проверим варианты?" + ✅ Показать туры / 📝 Изменить параметры

Бот: (3 карточки туров)

Пользователь: ✅ Выбрать этот тур

Бот: "Имя?" → "Фамилия?" → "Телефон?" → "Когда удобно?"

Бот: "Заявку передали менеджеру. Ожидайте."

Если хочешь — я в следующем сообщении соберу это в **короткое "ТЗ для разработки"** (блоки: состояния FSM, поля профиля, структура таблицы, форматы сообщений и кнопок) так, чтобы ты мог просто отдать это разработке/самому себе под aiogram.



да не плохо, а как будет происходить поиск в интернет, и еще делаем озвучку каждого сообщения в втеке подобрать с нуля

Ок, тогда делаем так: **поиск в интернете — только в ветке “Консультация”**, а в ветке **“Подобрать с нуля”** туры берём строго из Google Sheets, но каждое сообщение там получает кнопку 🗣️ Озвучить.

1) Как будет происходить “поиск в интернет” (ветка Консультация)

Логика (без магии)

1. Пользователь выбрал направление (или написал своё).
2. Бот задаёт 2–3 уточнения (даты/длительность/бюджет/кто едет).
3. Бот формирует **поисковые запросы** и делает **web-search** (через сервис/серверную часть).
4. Бот возвращает ответ **с опорой на найденное**:
 - “актуальные правила въезда / визы”
 - “сезонность и погода по месяцам”
 - “топ-локации/маршрут на N дней”
 - “примерный бюджет / на что реально уходит”
 - “что важно знать (безопасность/страховка/прививки)”
5. Дальше бот работает как консультант: пользователь задаёт вопросы → бот снова ищет, если нужно.

Какие кнопки в консультации

Inline под ответом:

- 🔍 Обновить инфу (повторный поиск)
- ❓ Задать вопрос
- 📅 Перейти к подбору туров (в таблицу)
- 📝 Оставить заявку
- 🗣️ Озвучить (опционально, но можно тоже)

Что именно бот “ищет”

Чтобы результаты были нормальные, делаем **3 типа источников**:

- Официальное: правила въезда/визы/ограничения
- Справочное: сезонность/погода/логистика
- Практика: маршруты, "что посмотреть", советы

И бот всегда строит ответ как "выжимка + 3–5 ссылок/источников" (пользователю можно показывать кратко).

2) Озвучка каждого сообщения в ветке "Подобрать с нуля"

Правило

В **каждом** сообщении анкеты и карточек тура добавляем inline:

- 🗣️ Озвучить
- (и рядом) 🏠 Меню / 🏠 Назад (если нужно)

Как это работает технически (концепт)

- При нажатии 🗣️ Озвучить бот берёт **текст последнего сообщения** (или заранее сохранённый "tts_text" для этого шага),
- отправляет в TTS (любая: OpenAI TTS / ElevenLabs / Google / Yandex),
- получает аудио-файл,
- отправляет пользователю voice/audio.

Важный нюанс (чтобы было красиво)

Текст для озвучки лучше хранить отдельно:

- на экране можно "красиво" с эмодзи и форматированием,
- а в TTS отправлять упрощённый текст без ссылок и мусора.

Например для карточки тура:

- сообщение: с пунктами + "Фото: ссылка"
 - tts_text: "Тур 'Непал. Трек к базовому лагерю'. Длительность 10 дней. Цена от ... Основные впечатления: ..."
-

3) Как совместить "подобрать с нуля" и "интернет"

Чтобы не путать:

- Ветка "Подобрать с нуля": поиск = **Google Sheets** (это твой каталог).
- Ветка "Консультация": поиск = **интернет** (актуальная инфа/советы/ответы).

Если вдруг в “подобрать” пользователь спросил: “а нужна ли виза?” — бот предлагает кнопку:

-  Перейти в консультацию (и там уже ищем в интернете)

4) Мини-ТЗ по кнопкам (итог)

Reply (статично)

-  Подобрать с нуля
-  Консультация
-  Меню
-  Назад

Inline (в “Подобрать с нуля” под каждым сообщением)

-  Озвучить
-  Изменить (где уместно)
-  Дальше /  Всё верно
-  Меню

Если хочешь, я сразу набросаю **конкретные “состояния” (FSM)** для обеих веток (шаги анкеты + шаги консультации + какие данные сохраняем), чтобы это прямо легло в код aiogram/n8n.



ага понял, я подумал и собрал финально такое тз, твоя задача вернуть мне код на языке PYTHON чтобы я вставил его в гугл коллаб в одну ячейку и запустил, токены все бери из серектов коллаба ни в коем случае не из окружений, бот должен быть написан на языке aiogram3х соблюдай всю документацию и правила написания для того чтобы не было конфликтов и ошибок
Вот ссылка на гугл таблицу
<https://docs.google.com/spreadsheets/d/1aypl3sQSexD6MP4kawiRN7pWWi3iL7frWPHNlmHy12s/edit?usp=sharing>

Пользователь нажимает /start. Бот пишет: "Привет! Я AI-тур-агент — помогу собрать заявку. У вас уже есть идея направления или подбираем с нуля?"

Реплай-клавиатура (показывается после /start):

"У меня есть идея"

"Подобрать с нуля"

Глобальная реплей-клавиатура, которая доступна дальше почти везде:

"Назад" — возвращает на предыдущий шаг (предыдущее состояние)

"Вернуться в начало" — сбрасывает сценарий и возвращает на приветствие /start

Сценарий 1: "У меня есть идея" (с консультацией и поиском в интернете)

Цель: быстро собрать минимальную анкету (направление, состав, бюджет, тип отдыха), затем включить режим консультации, после "Нет вопросов" показать анкету и собрать контакты.

Шаг 1. Направление (ввод текстом)

Бот спрашивает: "Куда хотите поехать? Напишите направление текстом."

Пользователь пишет текст (любое направление, не ограничиваем списком).

Шаг 2. Сколько взрослых (инлайн)

Сообщение: "Сколько взрослых?"

Инлайн-кнопки: 1, 2, 3, 4 / 5, 6, 7, 8 / "Ввести вручную"

Ограничение: взрослые строго 1–8.

Если "Ввести вручную" — пользователь вводит число, бот валидирует 1–8.

Шаг 3. Сколько детей (инлайн)

Сообщение: "Сколько детей?"

Инлайн-кнопки: 0, 1, 2 / 3, 4, 5 / "Ввести вручную"

Ограничение: дети строго 0–5.

Если "Ввести вручную" — пользователь вводит число, бот валидирует 0–5.

Если детей > 0:

Бот спрашивает текстом: "Укажите возраст каждого ребёнка через запятую (всего детей: N)."

Пользователь вводит N чисел, бот проверяет, что количество возрастов равно N.

Шаг 4. Бюджет (инлайн)

Сообщение: "Бюджет:"

Инлайн-кнопки: "До 50к", "50–100к", "100–200к", "200к+", "Ввести вручную"

Если вручную — пользователь вводит текстом (например "120 000 на двоих").

Шаг 5. Тип отдыха (инлайн)

Сообщение: "Тип отдыха:"

Инлайн-кнопки:

"Пляж/релакс", "Экскурсии", "Активный", "Семейный", "Природа/горы", "Круиз", "Ввести вручную"

Если вручную — пользователь вводит текстом.

Шаг 8. Включается режим консультации

После выбора типа отдыха бот включает "режим консультации" и пишет:

"Режим консультации включён. Задайте вопрос по поездке или нажмите «Нет вопросов»."

Только в этом месте появляется специальная реплей-клавиатура консультации:

"Нет вопросов"

"Назад"

"Вернуться в начало"

Как работает консультация (важное поведение)

Бот не отвечает сам "по своей инициативе" и не делает поиск сам. Он ждёт вопрос пользователя.

Если пользователь пишет вопрос:

Сначала проходит фильтр "вопрос про путешествия/тур или нет".

Если не про тур — бот отвечает: "Этот вопрос выходит за пределы консультации по туру/поездке." (без поиска).

Если про тур — бот формирует запрос к LLM с включённым инструментом `web_search` и отвечает кратко и практично.

Поиск в интернете выполняется только при реальном вопросе пользователя, и только в режиме консультации.

Как поиск происходит

Используется "`web_search`".

В запрос передаётся контекст анкеты (направление, состав, бюджет, тип отдыха) + текст вопроса пользователя.

Результат — текстовый ответ в чат.

Озвучка в ветке "У меня есть идея"

У каждого сообщения бота в этом сценарии автоматически добавляется инлайн-кнопка "🔊 Озвучить".

Нажатие "Озвучить" генерирует аудио (mp3) через OpenAI TTS и бот отправляет его как Audio в чат. Озвучка добавляется к каждому сообщению бота именно в ветке "У меня есть идея" (в том числе к ответам консультации).

Шаг 9. "Нет вопросов" (финал ветки "У меня есть идея")

Когда пользователь нажимает реплай "Нет вопросов":

Бот сразу отправляет анкету: Направление, состав, бюджет, тип (ничего не меняя).

В этом же сообщении бот просит контактные данные так же, как в ветке "Подобрать с нуля":

"Отправьте номер телефона (кнопкой) или напишите

вручную:"

и показывает реплей-клавиатуру отправки номера:

" Отправить номер"

"Назад"

"Вернуться в начало"

После получения номера бот просит: "Введите
Фамилию Имя:"

После ФИО бот выводит подтверждение:

"Спасибо, {ФИО}! Мы отправили вашу заявку
менеджеру — скоро с вами свяжутся для оплаты.
Телефон: ..."
(реальной отправки менеджеру нет — только текст,
что отправили)

Сценарий 2: "Подобрать с нуля" (подбор по базе
Google Sheets)

Цель: собрать ту же анкету только добавить бюджет и
звезды отеля, потом выбрать направление из
доступных в таблице, затем показать 1–несколько
вариантов туров (картинка + описание) и дать
пользователю выбрать, после чего показать анкету и
собрать контакты.

Шаги сбора анкеты (аналогично сценарию 1, без
консультации)

Взрослые (инлайн, 1–8, или вручную)

Дети (инлайн, 0–5, или вручную)

Возраст детей (если нужны)

Звёзды отеля (инлайн/вручную)

Даты (инлайн: точные/примерно, далее период
инлайн или вручную)

Бюджет (инлайн/вручную)

Тип отдыха (инлайн/вручную)

После пункта "Тип отдыха" начинается этап базы (Google Sheets).

Как устроена база и что берём из Google Sheets
Таблица содержит строки "туров". Минимальные поля:

"страна"

"URL" (ссылка на картинку/файл)
И дополнительное поле (новое):

"описание тура"

Важно: "страна" в таблице может повторяться (сафари, сафари 2, сафари 3 и т.п.). Бот нормализует "базовое" имя страны (убирает " 2/3" на конце) для группировки.

Этап 1. Выбор направления из доступных (инлайн, пагинация)

Бот показывает список направлений, которые реально есть в таблице, инлайн-кнопками (по 8 на страницу).

Кнопки навигации страниц: "◀" и "▶" (инлайн).
Пользователь выбирает направление.

Этап 2. Подбор туров внутри направления по описанию

После выбора направления бот берёт все строки (туры) из таблицы для этого направления и пытается подобрать "похожие" по описанию под анкету.
Правило: тур не обязан совпадать на 100% — бот обязан показать 1 или несколько приблизительно подходящих вариантов.

Этап 3. Показ вариантов и выбор пользователем

Бот отправляет варианты по 2 штуки за раз, если есть

только 1 вариант только его и отправляет:

фото (из URL) + подпись, где есть "Вариант тура #N"
и "Описание"

инлайн-кнопка на каждом варианте: "Выбрать этот тур"

Если вариантов много — бот дополнительно присылает инлайн-кнопку: "Показать ещё варианты"

Пользователь нажимает "Выбрать этот тур" —
выбранный тур сохраняется в анкете.

Этап 4. Анкета и подтверждение

Бот отправляет анкету, в которую включено:

все поля анкеты

плюс блок "Подобранный тур" (описание и ссылка на картинку)

Инлайн-кнопки:

 Подтвердить анкету"

 Изменить"

Изменение анкеты (важное правило)

Если пользователь нажимает " Изменить", бот показывает инлайн-список полей:

Направление

Состав

Звёзды отеля

Даты

Бюджет

Тип отдыха

(для сценария "Подобрать с нуля" дополнительно:

"Тур") Изменение доступно только в ветке
"Подобрать с нуля" в ветке "Есть идея" выводим
анкету сразу и просим номер и данные ФИО

После выбора поля бот меняет только это поле и не
просит заново остальные.

После изменения поля бот возвращает пользователя
к анкете (чтобы подтвердить).

Сбор контактов (в обеих ветках)

После подтверждения анкеты (или в ветке "У меня
есть идея" после "Нет вопросов" сразу):

Просим номер телефона:

реплай-кнопка " 📞 Отправить номер"
(request_contact)

или пользователь вводит ручную

Просим "Фамилию Имя" текстом

Финальное сообщение: "Мы отправили вашу заявку
менеджеру — скоро с вами свяжутся для оплаты"
(без реальной отправки)

Меню команд бота (Telegram command menu, не
реплай)

Команды доступны через "/" (встроенное меню
Telegram):

/help — кратко что делает бот

/contacts — "Менеджер: Алёна" и телефон
89998765432

/site — ссылка <https://neural-university.ru/>

Думал на протяжении 2m 10s >

python

 Копировать код

```

# =====
# AI Tour Agent (Telegram bot) – aiogram 3.x – Google Colab
# One-cell запуск. Токены берём ТОЛЬКО из Colab Secrets.
# Требуемые Secrets (в Colab -> "Secrets"):
#   TELEGRAM_BOT_TOKEN
#   OPENAI_API_KEY
# =====

!pip -q install "aiogram>=3.4,<4.0" "openai>=1.40.0" pandas requests nest_

import re
import os
import math
import json
import time
import asyncio
import tempfile
from dataclasses import dataclass
from typing import Any, Dict, List, Optional, Tuple

import pandas as pd
import requests
import nest_asyncio

from aiogram import Bot, Dispatcher, Router, F
from aiogram.types import (
    Message,
    CallbackQuery,
    ReplyKeyboardMarkup,
    KeyboardButton,
    InlineKeyboardButton,
)
from aiogram.utils.keyboard import InlineKeyboardBuilder
from aiogram.filters import Command, CommandStart
from aiogram.fsm.storage.memory import MemoryStorage
from aiogram.fsm.state import State, StatesGroup
from aiogram.fsm.context import FSMContext

from openai import OpenAI

nest_asyncio.apply()

# -----

```



```

# 1) Secrets (ONLY Colab)
# -----
try:
    from google.colab import userdata # type: ignore
except Exception as e:
    raise RuntimeError("Этот код рассчитан на Google Colab (нужен google.c

TELEGRAM_BOT_TOKEN = userdata.get("TELEGRAM_BOT_TOKEN")
OPENAI_API_KEY = userdata.get("OPENAI_API_KEY")

if not TELEGRAM_BOT_TOKEN:
    raise ValueError("Не найден секрет TELEGRAM_BOT_TOKEN в Colab Secrets.
if not OPENAI_API_KEY:
    raise ValueError("Не найден секрет OPENAI_API_KEY в Colab Secrets.")

# -----
# 2) Config
# -----
SHEET_URL = "https://docs.google.com/spreadsheets/d/1aypI3sQSexD6MP4kawiRN
SHEET_ID = re.search(r"/spreadsheets/d/([a-zA-Z0-9-_]+)", SHEET_URL).group

# Public CSV export (first sheet)
SHEET_CSV_URL = f"https://docs.google.com/spreadsheets/d/{SHEET_ID}/gviz/t

OPENAI_TTS_MODEL = "gpt-4o-mini-tts"
OPENAI_TTS_VOICE = "coral" # можно менять на alloy/ash/...
OPENAI_CHAT_MODEL = "gpt-5" # для web_search

# -----
# 3) Helpers – Google Sheets
# -----
_SHEET_CACHE: Dict[str, Any] = {"ts": 0, "df": None}

def _norm_col(s: str) -> str:
    return re.sub(r"\s+", " ", str(s)).strip().lower()

def _extract_drive_file_id(url: str) -> Optional[str]:
    if not url:
        return None
    m = re.search(r"/d/([a-zA-Z0-9-_]+)", url)
    if m:
        return m.group(1)
    m = re.search(r"[?&]id=([a-zA-Z0-9-_]+)", url)

```

```
if m:
    return m.group(1)
return None

def _to_direct_url(url: str) -> str:
    if not url:
        return url
    fid = _extract_drive_file_id(url)
    if fid:
        return f"https://drive.google.com/uc?export=download&id={fid}"
    return url

def load_sheet_df(force: bool = False, ttl_sec: int = 300) -> pd.DataFrame:
    now = time.time()
    if not force and _SHEET_CACHE["df"] is not None and (now - _SHEET_CACH
        return _SHEET_CACHE["df"]

    resp = requests.get(SHEET_CSV_URL, timeout=20)
    resp.raise_for_status()
    df = pd.read_csv(pd.io.common.StringIO(resp.text))

    # Normalize columns
    cols = {_norm_col(c): c for c in df.columns}
    # Expected: "страна", "url", "описание типа"
    country_col = cols.get("страна") or next((cols[k] for k in cols if "ст
    url_col = cols.get("url") or next((cols[k] for k in cols if "url" in k
    desc_col = cols.get("описание типа") or next((cols[k] for k in cols if

    if not country_col or not url_col:
        raise ValueError(
            "В таблице не найдены обязательные колонки 'страна' и 'URL'. "
            f"Найденные колонки: {list(df.columns)}"
        )

    df = df.rename(columns={country_col: "страна", url_col: "URL"})
    if desc_col:
        df = df.rename(columns={desc_col: "описание типа"})
    else:
        df["описание типа"] = ""

    # Clean
    df["страна"] = df["страна"].astype(str).fillna("").str.strip()
    df["URL"] = df["URL"].astype(str).fillna("").str.strip()
```

```

df["описание типа"] = df["описание типа"].astype(str).fillna("").str.s

_SHEET_CACHE["ts"] = now
_SHEET_CACHE["df"] = df
return df

def base_country_name(s: str) -> str:
    s = re.sub(r"\s+", " ", str(s)).strip()
    s = re.sub(r"\s+\d+$", "", s).strip() # remove trailing " 2/3"
    return s

def get_available_directions() -> List[str]:
    df = load_sheet_df()
    bases = sorted({base_country_name(x) for x in df["страна"].tolist() if
    return bases

def get_tours_for_direction(direction_base: str) -> List[Dict[str, str]]:
    df = load_sheet_df()
    rows = df.to_dict(orient="records")
    tours = []
    for r in rows:
        c = base_country_name(r.get("страна", ""))
        if c.lower() == direction_base.lower():
            tours.append({
                "страна": str(r.get("страна", "")).strip(),
                "URL": _to_direct_url(str(r.get("URL", "")).strip()),
                "описание типа": str(r.get("описание типа", "")).strip(),
            })
    return tours

# -----
# 4) Helpers – scoring tours (simple similarity)
# -----
_RU_WORD = re.compile(r"[a-zA-Za-яA-ЯёЁ0-9]+")

def tokenize(text: str) -> List[str]:
    text = (text or "").lower()
    return _RU_WORD.findall(text)

def jaccard_score(a: str, b: str) -> float:
    A = set(tokenize(a))
    B = set(tokenize(b))
    if not A and not B:

```

```

        return 0.0
    if not A or not B:
        return 0.0
    return len(A & B) / (len(A | B) + 1e-9)

def build_zero_profile_text(data: Dict[str, Any]) -> str:
    parts = []
    parts.append(f"взрослые: {data.get('adults')}")
    parts.append(f"дети: {data.get('kids')}")
    if data.get("kids_ages"):
        parts.append(f"возраст детей: {data.get('kids_ages')}")
    parts.append(f"звезды: {data.get('stars')}")
    parts.append(f"даты: {data.get('dates')}")
    parts.append(f"бюджет: {data.get('budget')}")
    parts.append(f"тип отдыха: {data.get('rest_type')}")
    return ", ".join([p for p in parts if p and "None" not in str(p)])

def rank_tours(profile_text: str, tours: List[Dict[str, str]]) -> List[Tup
    scored = []
    for i, t in enumerate(tours):
        desc = t.get("описание тура", "") or ""
        score = jaccard_score(profile_text, desc)
        scored.append((i, score))
    scored.sort(key=lambda x: x[1], reverse=True)
    return scored

# -----
# 5) OpenAI helpers (TTS + web_search)
# -----
oai = OpenAI(api_key=OPENAI_API_KEY)

async def openai_tts_mp3_bytes(text: str) -> bytes:
    text = (text or "").strip()
    if not text:
        text = " "
    # limit 4096 chars (API limit)
    text = text[:4096]

    with tempfile.NamedTemporaryFile(suffix=".mp3", delete=False) as tmp:
        tmp_path = tmp.name

    try:
        # streaming to file (official)

```

```

with oai.audio.speech.with_streaming_response.create(
    model=OPENAI_TTS_MODEL,
    voice=OPENAI_TTS_VOICE,
    input=text,
    instructions="Говори чётко и дружелюбно. Это голос ИИ."
) as response:
    response.stream_to_file(tmp_path)

with open(tmp_path, "rb") as f:
    return f.read()
finally:
    try:
        os.remove(tmp_path)
    except Exception:
        pass

def looks_like_travel_question(text: str) -> bool:
    t = (text or "").lower()
    if len(t) < 3:
        return False
    # simple heuristic filter
    kw = [
        "виза", "паспорт", "перелет", "самолет", "аэропорт", "багаж", "погода",
        "звезды", "страховка", "прививка", "маршрут", "что посмотреть", "достопримечательности",
        "бюджет", "трансфер", "экскурс", "тур", "поездка", "пляж", "горы", "сафари",
        "безопасность", "сим-карта", "связь", "интернет", "розетка", "время", "еда",
    ]
    return any(k in t for k in kw) or "?" in t

async def openai_answer_with_web_search(profile: str, question: str) -> str:
    prompt = (
        "Ты – туристический консультант. Отвечай кратко, практично, по делу.\n"
        "Если данных недостаточно – задай 1 уточняющий вопрос в конце.\n\n"
        f"Анкета клиента: {profile}\n"
        f"Вопрос клиента: {question}\n\n"
        "Используй web_search ТОЛЬКО чтобы получить актуальную информацию.\n"
        "Если отвечаешь фактами (правила въезда и т.п.) – оставь встроенными.\n"
    )
    resp = oai.responses.create(
        model=OPENAI_CHAT_MODEL,
        tools=[{"type": "web_search"}],
        input=prompt,
        reasoning={"effort": "low"},
    )

```

```

    )
    return (resp.output_text or "").strip() or "Не смог сформировать ответ"

# -----
# 6) Keyboards
# -----
KB_START = ReplyKeyboardMarkup(
    keyboard=[
        [KeyboardButton(text="У меня есть идея")],
        [KeyboardButton(text="Подобрать с нуля")],
    ],
    resize_keyboard=True,
    selective=True,
)

KB_GLOBAL = ReplyKeyboardMarkup(
    keyboard=[
        [KeyboardButton(text="Назад"), KeyboardButton(text="Вернуться в на
    ],
    resize_keyboard=True,
    selective=True,
)

KB_CONSULT = ReplyKeyboardMarkup(
    keyboard=[
        [KeyboardButton(text="Нет вопросов")],
        [KeyboardButton(text="Назад"), KeyboardButton(text="Вернуться в на
    ],
    resize_keyboard=True,
    selective=True,
)

def kb_contact_request() -> ReplyKeyboardMarkup:
    return ReplyKeyboardMarkup(
        keyboard=[
            [KeyboardButton(text="📞 Отправить номер", request_contact=True
            [KeyboardButton(text="Назад"), KeyboardButton(text="Вернуться
        ],
        resize_keyboard=True,
        selective=True,
    )

```

```

def add_tts_button(builder: InlineKeyboardBuilder, payload_key: str) -> In

```

```
builder.row(InlineKeyboardButton(text="🔊 Озвучить", callback_data=f"t
return builder
```

```
# -----
# 7) FSM States
# -----
class IdeaFlow(StatesGroup):
    direction = State()
    adults = State()
    adults_manual = State()
    kids = State()
    kids_manual = State()
    kids_ages = State()
    budget = State()
    budget_manual = State()
    rest_type = State()
    rest_type_manual = State()
    consult = State()
    contact_phone = State()
    contact_name = State()

class ZeroFlow(StatesGroup):
    adults = State()
    adults_manual = State()
    kids = State()
    kids_manual = State()
    kids_ages = State()
    stars = State()
    stars_manual = State()
    dates_mode = State()
    dates_manual = State()
    budget = State()
    budget_manual = State()
    rest_type = State()
    rest_type_manual = State()
    choose_direction = State()
    show_tours = State()
    summary = State()
    edit_field = State()
    contact_phone = State()
    contact_name = State()

# -----
```

```

# 8) Back stack helpers
# -----
async def push_state(ctx: FSMContext, state: State):
    data = await ctx.get_data()
    hist = data.get("history", [])
    st = await ctx.get_state()
    if st:
        hist.append(st)
    await ctx.update_data(history=hist)
    await ctx.set_state(state)

async def go_back(ctx: FSMContext) -> Optional[str]:
    data = await ctx.get_data()
    hist = data.get("history", [])
    if not hist:
        return None
    prev = hist.pop()
    await ctx.update_data(history=hist)
    await ctx.set_state(prev)
    return prev

async def reset_to_start(ctx: FSMContext):
    await ctx.clear()

# -----
# 9) TTS payload store (per chat)
# -----
# payload_key -> {"chat_id": int, "text": str}
TTS_STORE: Dict[str, Dict[str, Any]] = {}

def make_tts_key(chat_id: int, message_id: int) -> str:
    return f"{chat_id}:{message_id}"

async def register_tts_text(chat_id: int, msg: Message, text_for_tts: str)
    key = make_tts_key(chat_id, msg.message_id)
    TTS_STORE[key] = {"chat_id": chat_id, "text": text_for_tts}
    return key

# -----
# 10) Rendering prompts
# -----
router = Router()

```



```

async def send_idea(bot: Bot, chat_id: int, text: str, ctx: FSMContext, bu
    if builder is None:
        builder = InlineKeyboardBuilder()
        # placeholder key; we will update after sending (needs message_id)
        msg = await bot.send_message(chat_id, text, reply_markup=builder.as_ma
        key = await register_tts_text(chat_id, msg, text)
        # re-send markup with tts button appended (edit)
        b = builder
        add_tts_button(b, key)
        await bot.edit_message_reply_markup(chat_id, msg.message_id, reply_mar
        return msg

```

```

async def send_idea_plain(bot: Bot, chat_id: int, text: str, ctx: FSMConte
    b = InlineKeyboardBuilder()
    msg = await bot.send_message(chat_id, text)
    key = await register_tts_text(chat_id, msg, text)
    add_tts_button(b, key)
    await bot.edit_message_reply_markup(chat_id, msg.message_id, reply_mar
    return msg

```

```

def adults_inline(prefix: str) -> InlineKeyboardBuilder:
    b = InlineKeyboardBuilder()
    for n in [1,2,3,4,5,6,7,8]:
        b.button(text=str(n), callback_data=f"{prefix}:{n}")
    b.adjust(4,4)
    b.row(InlineKeyboardButton(text="Ввести вручную", callback_data=f"{pre
    return b

```

```

def kids_inline(prefix: str) -> InlineKeyboardBuilder:
    b = InlineKeyboardBuilder()
    for n in [0,1,2,3,4,5]:
        b.button(text=str(n), callback_data=f"{prefix}:{n}")
    b.adjust(3,3)
    b.row(InlineKeyboardButton(text="Ввести вручную", callback_data=f"{pre
    return b

```

```

def budget_inline(prefix: str) -> InlineKeyboardBuilder:
    b = InlineKeyboardBuilder()
    b.button(text="До 50к", callback_data=f"{prefix}:До 50к")
    b.button(text="50-100к", callback_data=f"{prefix}:50-100к")
    b.button(text="100-200к", callback_data=f"{prefix}:100-200к")
    b.button(text="200к+", callback_data=f"{prefix}:200к+")
    b.adjust(2,2)

```

```

b.row(InlineKeyboardButton(text="Ввести вручную", callback_data=f"{pre
return b

def rest_type_inline(prefix: str) -> InlineKeyboardBuilder:
    b = InlineKeyboardBuilder()
    opts = ["Пляж/релакс", "Экскурсии", "Активный", "Семейный", "Природа/горы"]
    for o in opts:
        b.button(text=o, callback_data=f"{prefix}:{o}")
    b.adjust(2,2,2)
    b.row(InlineKeyboardButton(text="Ввести вручную", callback_data=f"{pre
return b

def stars_inline(prefix: str) -> InlineKeyboardBuilder:
    b = InlineKeyboardBuilder()
    for o in ["Любые", "3★", "4★", "5★"]:
        b.button(text=o, callback_data=f"{prefix}:{o}")
    b.adjust(2,2)
    b.row(InlineKeyboardButton(text="Ввести вручную", callback_data=f"{pre
return b

def dates_mode_inline(prefix: str) -> InlineKeyboardBuilder:
    b = InlineKeyboardBuilder()
    b.button(text="Точные", callback_data=f"{prefix}:exact")
    b.button(text="Примерно", callback_data=f"{prefix}:approx")
    b.adjust(2)
    b.row(InlineKeyboardButton(text="Ввести вручную", callback_data=f"{pre
return b

def dates_approx_inline(prefix: str) -> InlineKeyboardBuilder:
    b = InlineKeyboardBuilder()
    opts = ["В этом месяце", "В следующем месяце", "Через 2–3 месяца", "Лето"]
    for o in opts:
        b.button(text=o, callback_data=f"{prefix}:{o}")
    b.adjust(2,2,3)
    b.row(InlineKeyboardButton(text="Ввести вручную", callback_data=f"{pre
return b

def paginate_buttons(items: List[str], page: int, per_page: int, cb_prefix
    b = InlineKeyboardBuilder()
    start = page * per_page
    chunk = items[start:start+per_page]
    for it in chunk:
        b.button(text=it, callback_data=f"{cb_prefix}:pick:{it}")

```

```

b.adjust(2,2,2,2)
nav = InlineKeyboardBuilder()
if page > 0:
    nav.button(text="⬅️", callback_data=f"{cb_prefix}:page:{page-1}")
if start + per_page < len(items):
    nav.button(text="➡️", callback_data=f"{cb_prefix}:page:{page+1}")
if nav.buttons:
    b.row(*nav.buttons[0])
return b

# -----
# 11) Common handlers: /help /contacts /site
# -----
@router.message(Command("help"))
async def cmd_help(m: Message):
    await m.answer(
        "Я AI-тип-агент: помогу собрать заявку, проконсультировать по напр
        "Команды:\n"
        "/start – начать заново\n"
        "/help – помощь\n"
        "/contacts – контакты менеджера\n"
        "/site – сайт\n\n"
        "🗣️ Озвучка – это голос ИИ.",
        reply_markup=KB_START
    )

@router.message(Command("contacts"))
async def cmd_contacts(m: Message):
    await m.answer("Менеджер: Алёна\nТелефон: 89998765432")

@router.message(Command("site"))
async def cmd_site(m: Message):
    await m.answer("https://neural-university.ru/")

# -----
# 12) Global: /start + reset + back
# -----
@router.message(CommandStart())
async def start(m: Message, state: FSMContext):
    await reset_to_start(state)
    await m.answer(
        "Привет! Я AI-тип-агент – помогу собрать заявку. У вас уже есть ид
        reply_markup=KB_START

```

```
)
```

```
@router.message(F.text == "Вернуться в начало")
async def go_home(m: Message, state: FSMContext):
    await reset_to_start(state)
    await m.answer(
        "Привет! Я AI-тур-агент – помогу собрать заявку. У вас уже есть ид
        reply_markup=KB_START
    )
```

```
@router.message(F.text == "Назад")
async def back(m: Message, state: FSMContext, bot: Bot):
    prev = await go_back(state)
    if not prev:
        await m.answer("Вы уже в начале. Нажмите «Вернуться в начало».", r
        return
    # Re-render based on state
    st = await state.get_state()
    chat_id = m.chat.id

    # IDEA
    if st == IdeaFlow.direction.state:
        await send_idea_plain(bot, chat_id, "Куда хотите поехать? Напишите
        await m.answer(" ", reply_markup=KB_GLOBAL)
    elif st == IdeaFlow.adults.state:
        b = adults_inline("idea_adults")
        await send_idea(bot, chat_id, "Сколько взрослых?", state, b)
        await m.answer(" ", reply_markup=KB_GLOBAL)
    elif st == IdeaFlow.kids.state:
        b = kids_inline("idea_kids")
        await send_idea(bot, chat_id, "Сколько детей?", state, b)
        await m.answer(" ", reply_markup=KB_GLOBAL)
    elif st == IdeaFlow.kids_ages.state:
        data = await state.get_data()
        n = data.get("kids", 0)
        await send_idea_plain(bot, chat_id, f"Укажите возраст каждого ребё
        await m.answer(" ", reply_markup=KB_GLOBAL)
    elif st == IdeaFlow.budget.state:
        b = budget_inline("idea_budget")
        await send_idea(bot, chat_id, "Бюджет:", state, b)
        await m.answer(" ", reply_markup=KB_GLOBAL)
    elif st == IdeaFlow.rest_type.state:
        b = rest_type_inline("idea_type")
```

```
    await send_idea(bot, chat_id, "Тип отдыха:", state, b)
    await m.answer(" ", reply_markup=KB_GLOBAL)
elif st == IdeaFlow.consult.state:
    await send_idea_plain(bot, chat_id, "Режим консультации включён. 3
    await m.answer(" ", reply_markup=KB_CONSULT)

# ZERO
elif st == ZeroFlow.adults.state:
    b = adults_inline("zero_adults")
    await m.answer("Сколько взрослых?", reply_markup=KB_GLOBAL)
    await m.answer("Выберите:", reply_markup=b.as_markup())
elif st == ZeroFlow.kids.state:
    b = kids_inline("zero_kids")
    await m.answer("Сколько детей?", reply_markup=KB_GLOBAL)
    await m.answer("Выберите:", reply_markup=b.as_markup())
elif st == ZeroFlow.kids_ages.state:
    data = await state.get_data()
    n = data.get("kids", 0)
    await m.answer(f"Укажите возраст каждого ребёнка через запятую (вс
elif st == ZeroFlow.stars.state:
    b = stars_inline("zero_stars")
    await m.answer("Звёзды отеля:", reply_markup=KB_GLOBAL)
    await m.answer("Выберите:", reply_markup=b.as_markup())
elif st == ZeroFlow.dates_mode.state:
    b = dates_mode_inline("zero_dates")
    await m.answer("Даты поездки:", reply_markup=KB_GLOBAL)
    await m.answer("Выберите:", reply_markup=b.as_markup())
elif st == ZeroFlow.budget.state:
    b = budget_inline("zero_budget")
    await m.answer("Бюджет:", reply_markup=KB_GLOBAL)
    await m.answer("Выберите:", reply_markup=b.as_markup())
elif st == ZeroFlow.rest_type.state:
    b = rest_type_inline("zero_type")
    await m.answer("Тип отдыха:", reply_markup=KB_GLOBAL)
    await m.answer("Выберите:", reply_markup=b.as_markup())
elif st == ZeroFlow.choose_direction.state:
    data = await state.get_data()
    page = int(data.get("dir_page", 0))
    dirs = get_available_directions()
    b = paginate_buttons(dirs, page, 8, "zero_dir")
    await m.answer("Выберите направление из базы:", reply_markup=KB_GL
    await m.answer("Направления:", reply_markup=b.as_markup())
elif st == ZeroFlow.show_tours.state:
```

```

        await m.answer("Продолжаем показ вариантов. Нажмите «Показать ещё»")
    elif st == ZeroFlow.summary.state:
        await render_zero_summary(m, state)
    elif st == ZeroFlow.contact_phone.state:
        await m.answer("Отправьте номер телефона (кнопкой) или напишите в чат")
    elif st == IdeaFlow.contact_phone.state:
        await m.answer("Отправьте номер телефона (кнопкой) или напишите в чат")
    else:
        await m.answer("Ок, вернул назад.", reply_markup=KB_GLOBAL)

# -----
# 13) Branch selection
# -----
@router.message(F.text == "У меня есть идея")
async def choose_idea(m: Message, state: FSMContext, bot: Bot):
    await reset_to_start(state)
    await state.update_data(flow="idea", history=[])
    await push_state(state, IdeaFlow.direction)
    await send_idea_plain(bot, m.chat.id, "Куда хотите поехать? Напишите название")
    await m.answer("", reply_markup=KB_GLOBAL)

@router.message(F.text == "Подобрать с нуля")
async def choose_zero(m: Message, state: FSMContext):
    await reset_to_start(state)
    await state.update_data(flow="zero", history=[])
    await push_state(state, ZeroFlow.adults)
    b = adults_inline("zero_adults")
    await m.answer("Сколько взрослых?", reply_markup=KB_GLOBAL)
    await m.answer("Выберите:", reply_markup=b.as_markup())

# -----
# 14) IDEA flow handlers
# -----
@router.message(IdeaFlow.direction)
async def idea_direction(m: Message, state: FSMContext, bot: Bot):
    txt = (m.text or "").strip()
    if not txt:
        await send_idea_plain(bot, m.chat.id, "Напишите направление текста")
        return
    await state.update_data(direction=txt)
    await push_state(state, IdeaFlow.adults)
    b = adults_inline("idea_adults")
    await send_idea(bot, m.chat.id, "Сколько взрослых?", state, b)

```

```
await m.answer(" ", reply_markup=KB_GLOBAL)
```

```
@router.callback_query(F.data.startswith("idea_adults:"))
```

```
async def idea_adults_cb(c: CallbackQuery, state: FSMContext, bot: Bot):
```

```
    await c.answer()
```

```
    val = c.data.split(":", 1)[1]
```

```
    if val == "manual":
```

```
        await push_state(state, IdeaFlow.adults_manual)
```

```
        await send_idea_plain(bot, c.message.chat.id, "Введите число взрос.
```

```
        return
```

```
    n = int(val)
```

```
    if not (1 <= n <= 8):
```

```
        await send_idea_plain(bot, c.message.chat.id, "Взрослые должны быт
```

```
        return
```

```
    await state.update_data(adults=n)
```

```
    await push_state(state, IdeaFlow.kids)
```

```
    b = kids_inline("idea_kids")
```

```
    await send_idea(bot, c.message.chat.id, "Сколько детей?", state, b)
```

```
    await bot.send_message(c.message.chat.id, " ", reply_markup=KB_GLOBAL)
```

```
@router.message(IdeaFlow.adults_manual)
```

```
async def idea_adults_manual(m: Message, state: FSMContext, bot: Bot):
```

```
    txt = (m.text or "").strip()
```

```
    if not txt.isdigit():
```

```
        await send_idea_plain(bot, m.chat.id, "Нужно число 1–8. Введите ещ
```

```
        return
```

```
    n = int(txt)
```

```
    if not (1 <= n <= 8):
```

```
        await send_idea_plain(bot, m.chat.id, "Взрослые строго 1–8. Вводит
```

```
        return
```

```
    await state.update_data(adults=n)
```

```
    await push_state(state, IdeaFlow.kids)
```

```
    b = kids_inline("idea_kids")
```

```
    await send_idea(bot, m.chat.id, "Сколько детей?", state, b)
```

```
    await m.answer(" ", reply_markup=KB_GLOBAL)
```

```
@router.callback_query(F.data.startswith("idea_kids:"))
```

```
async def idea_kids_cb(c: CallbackQuery, state: FSMContext, bot: Bot):
```

```
    await c.answer()
```

```
    val = c.data.split(":", 1)[1]
```

```
    if val == "manual":
```

```
        await push_state(state, IdeaFlow.kids_manual)
```

```
        await send_idea_plain(bot, c.message.chat.id, "Введите число детей
```

```

        return
    k = int(val)
    if not (0 <= k <= 5):
        await send_idea_plain(bot, c.message.chat.id, "Дети должны быть 0–5")
        return
    await state.update_data(kids=k)
    if k > 0:
        await push_state(state, IdeaFlow.kids_ages)
        await send_idea_plain(bot, c.message.chat.id, f"Укажите возраст ка")
    else:
        await push_state(state, IdeaFlow.budget)
        b = budget_inline("idea_budget")
        await send_idea(bot, c.message.chat.id, "Бюджет:", state, b)
        await bot.send_message(c.message.chat.id, " ", reply_markup=KB_GL0

```

```
@router.message(IdeaFlow.kids_manual)
```

```

async def idea_kids_manual(m: Message, state: FSMContext, bot: Bot):
    txt = (m.text or "").strip()
    if not txt.isdigit():
        await send_idea_plain(bot, m.chat.id, "Нужно число 0–5. Введите ещ")
        return
    k = int(txt)
    if not (0 <= k <= 5):
        await send_idea_plain(bot, m.chat.id, "Дети строго 0–5. Введите ещ")
        return
    await state.update_data(kids=k)
    if k > 0:
        await push_state(state, IdeaFlow.kids_ages)
        await send_idea_plain(bot, m.chat.id, f"Укажите возраст каждого ре")
    else:
        await push_state(state, IdeaFlow.budget)
        b = budget_inline("idea_budget")
        await send_idea(bot, m.chat.id, "Бюджет:", state, b)
        await m.answer(" ", reply_markup=KB_GLOBAL)

```

```
@router.message(IdeaFlow.kids_ages)
```

```

async def idea_kids_ages(m: Message, state: FSMContext, bot: Bot):
    data = await state.get_data()
    k = int(data.get("kids", 0))
    parts = [p.strip() for p in (m.text or "").split(",") if p.strip()]
    if len(parts) != k:
        await send_idea_plain(bot, m.chat.id, f"Нужно указать ровно {k} во")
        return

```



```

ages = []
for p in parts:
    if not re.fullmatch(r"\d{1,2}", p):
        await send_idea_plain(bot, m.chat.id, "Возраст должен быть числoм")
        return
    ages.append(int(p))
await state.update_data(kids_ages=ages)
await push_state(state, IdeaFlow.budget)
b = budget_inline("idea_budget")
await send_idea(bot, m.chat.id, "Бюджет:", state, b)
await m.answer(" ", reply_markup=KB_GLOBAL)

@router.callback_query(F.data.startswith("idea_budget:"))
async def idea_budget_cb(c: CallbackQuery, state: FSMContext, bot: Bot):
    await c.answer()
    val = c.data.split(":", 1)[1]
    if val == "manual":
        await push_state(state, IdeaFlow.budget_manual)
        await send_idea_plain(bot, c.message.chat.id, "Введите бюджет текстом")
        return
    await state.update_data(budget=val)
    await push_state(state, IdeaFlow.rest_type)
    b = rest_type_inline("idea_type")
    await send_idea(bot, c.message.chat.id, "Тип отдыха:", state, b)
    await bot.send_message(c.message.chat.id, " ", reply_markup=KB_GLOBAL)

@router.message(IdeaFlow.budget_manual)
async def idea_budget_manual(m: Message, state: FSMContext, bot: Bot):
    txt = (m.text or "").strip()
    if not txt:
        await send_idea_plain(bot, m.chat.id, "Введите бюджет текстом:", state)
        return
    await state.update_data(budget=txt)
    await push_state(state, IdeaFlow.rest_type)
    b = rest_type_inline("idea_type")
    await send_idea(bot, m.chat.id, "Тип отдыха:", state, b)
    await m.answer(" ", reply_markup=KB_GLOBAL)

@router.callback_query(F.data.startswith("idea_type:"))
async def idea_type_cb(c: CallbackQuery, state: FSMContext, bot: Bot):
    await c.answer()
    val = c.data.split(":", 1)[1]
    if val == "manual":

```

```

        await push_state(state, IdeaFlow.rest_type_manual)
        await send_idea_plain(bot, c.message.chat.id, "Введите тип отдыха")
        return
    await state.update_data(rest_type=val)
    await push_state(state, IdeaFlow.consult)
    await send_idea_plain(bot, c.message.chat.id, "Режим консультации вклю")
    await bot.send_message(c.message.chat.id, " ", reply_markup=KB_CONSULT

```

```
@router.message(IdeaFlow.rest_type_manual)
```

```

async def idea_type_manual(m: Message, state: FSMContext, bot: Bot):
    txt = (m.text or "").strip()
    if not txt:
        await send_idea_plain(bot, m.chat.id, "Введите тип отдыха текстом:")
        return
    await state.update_data(rest_type=txt)
    await push_state(state, IdeaFlow.consult)
    await send_idea_plain(bot, m.chat.id, "Режим консультации включён. Зад")
    await m.answer(" ", reply_markup=KB_CONSULT)

```

```
@router.message(IdeaFlow.consult, F.text == "Нет вопросов")
```

```

async def idea_no_questions(m: Message, state: FSMContext, bot: Bot):
    data = await state.get_data()
    direction = data.get("direction")
    adults = data.get("adults")
    kids = data.get("kids")
    kids_ages = data.get("kids_ages")
    budget = data.get("budget")
    rest_type = data.get("rest_type")

    ank = [
        f"📌 Анкета:",
        f"Направление: {direction}",
        f"Состав: взрослые {adults}, дети {kids}" + (f" (возраст: {kids_ag")
        f"Бюджет: {budget}",
        f"Тип отдыха: {rest_type}",
        "",
        "Отправьте номер телефона (кнопкой) или напишите вручную:"
    ]

    await push_state(state, IdeaFlow.contact_phone)
    await send_idea_plain(bot, m.chat.id, "\n".join(ank), state)
    await m.answer(" ", reply_markup=kb_contact_request())

```

```
@router.message(IdeaFlow.consult)
```

```

async def idea_consult_question(m: Message, state: FSMContext, bot: Bot):
    # ignore if user pressed navigation
    if (m.text or "").strip() in {"Назад", "Вернуться в начало"}:
        return

    q = (m.text or "").strip()
    if not q:
        await send_idea_plain(bot, m.chat.id, "Напишите вопрос текстом или
        return

    if not looks_like_travel_question(q):
        await send_idea_plain(bot, m.chat.id, "Этот вопрос выходит за пред
        return

    data = await state.get_data()
    profile = f"направление: {data.get('direction')}, взрослые: {data.get(
    try:
        answer = await openai_answer_with_web_search(profile, q)
    except Exception as e:
        answer = f"Не получилось выполнить поиск/ответ: {e}"
    await send_idea_plain(bot, m.chat.id, answer, state)
    await m.answer(" ", reply_markup=KB_CONSULT)

# IDEA contacts
def normalize_phone(s: str) -> Optional[str]:
    digits = re.sub(r"\D+", "", s or "")
    if len(digits) < 10:
        return None
    return digits

@router.message(IdeaFlow.contact_phone)
async def idea_phone(m: Message, state: FSMContext, bot: Bot):
    phone = None
    if m.contact and m.contact.phone_number:
        phone = normalize_phone(m.contact.phone_number)
    else:
        phone = normalize_phone(m.text or "")
    if not phone:
        await send_idea_plain(bot, m.chat.id, "Не распознал телефон. Отпра
        await m.answer(" ", reply_markup=kb_contact_request())
        return
    await state.update_data(phone=phone)
    await push_state(state, IdeaFlow.contact_name)

```

```

await send_idea_plain(bot, m.chat.id, "Введите Фамилию Имя:", state)
await m.answer(" ", reply_markup=KB_GLOBAL)

```

```
@router.message(IdeaFlow.contact_name)
```

```
async def idea_name(m: Message, state: FSMContext, bot: Bot):
```

```
    fio = re.sub(r"\s+", " ", (m.text or "").strip())
```

```
    if len(fio.split()) < 2:
```

```
        await send_idea_plain(bot, m.chat.id, "Нужно в формате «Фамилия Им
        return

```

```
    data = await state.get_data()
```

```
    phone = data.get("phone", "")
```

```
    msg = f"Спасибо, {fio}! Мы отправили вашу заявку менеджеру – скоро с в
```

```
    await send_idea_plain(bot, m.chat.id, msg, state)
```

```
    await reset_to_start(state)
```

```
    await m.answer(" ", reply_markup=KB_START)
```

```
# -----
```

```
# 15) ZERO flow handlers
```

```
# -----
```

```
@router.callback_query(F.data.startswith("zero_adults:"))
```

```
async def zero_adults_cb(c: CallbackQuery, state: FSMContext):
```

```
    await c.answer()
```

```
    val = c.data.split(":", 1)[1]
```

```
    if val == "manual":
```

```
        await push_state(state, ZeroFlow.adults_manual)
```

```
        await c.message.answer("Введите число взрослых (1–8):", reply_mark
        return

```

```
    n = int(val)
```

```
    if not (1 <= n <= 8):
```

```
        await c.message.answer("Взрослые строго 1–8. Выберите снова.", rep
        return

```

```
    await state.update_data(adults=n)
```

```
    await push_state(state, ZeroFlow.kids)
```

```
    b = kids_inline("zero_kids")
```

```
    await c.message.answer("Сколько детей?", reply_markup=KB_GLOBAL)
```

```
    await c.message.answer("Выберите:", reply_markup=b.as_markup())
```

```
@router.message(ZeroFlow.adults_manual)
```

```
async def zero_adults_manual(m: Message, state: FSMContext):
```

```
    txt = (m.text or "").strip()
```

```
    if not txt.isdigit():
```

```
        await m.answer("Нужно число 1–8. Введите ещё раз:", reply_markup=K
        return

```

```
n = int(txt)
if not (1 <= n <= 8):
    await m.answer("Взрослые строго 1–8. Введите ещё раз:", reply_mark
    return
await state.update_data(adults=n)
await push_state(state, ZeroFlow.kids)
b = kids_inline("zero_kids")
await m.answer("Сколько детей?", reply_markup=KB_GLOBAL)
await m.answer("Выберите:", reply_markup=b.as_markup())

@router.callback_query(F.data.startswith("zero_kids:"))
async def zero_kids_cb(c: CallbackQuery, state: FSMContext):
    await c.answer()
    val = c.data.split(":", 1)[1]
    if val == "manual":
        await push_state(state, ZeroFlow.kids_manual)
        await c.message.answer("Введите число детей (0–5):", reply_markup=
        return
    k = int(val)
    if not (0 <= k <= 5):
        await c.message.answer("Дети строго 0–5. Выберите снова.", reply_m
        return
    await state.update_data(kids=k)
    if k > 0:
        await push_state(state, ZeroFlow.kids_ages)
        await c.message.answer(f"Укажите возраст каждого ребёнка через зап
    else:
        await push_state(state, ZeroFlow.stars)
        b = stars_inline("zero_stars")
        await c.message.answer("Звёзды отеля:", reply_markup=KB_GLOBAL)
        await c.message.answer("Выберите:", reply_markup=b.as_markup())

@router.message(ZeroFlow.kids_manual)
async def zero_kids_manual(m: Message, state: FSMContext):
    txt = (m.text or "").strip()
    if not txt.isdigit():
        await m.answer("Нужно число 0–5. Введите ещё раз:", reply_markup=K
        return
    k = int(txt)
    if not (0 <= k <= 5):
        await m.answer("Дети строго 0–5. Введите ещё раз:", reply_markup=K
        return
    await state.update_data(kids=k)
```

```
if k > 0:
    await push_state(state, ZeroFlow.kids_ages)
    await m.answer(f"Укажите возраст каждого ребёнка через запятую (вс
else:
    await push_state(state, ZeroFlow.stars)
    b = stars_inline("zero_stars")
    await m.answer("Звёзды отеля:", reply_markup=KB_GLOBAL)
    await m.answer("Выберите:", reply_markup=b.as_markup())

@router.message(ZeroFlow.kids_ages)
async def zero_kids_ages(m: Message, state: FSMContext):
    data = await state.get_data()
    k = int(data.get("kids", 0))
    parts = [p.strip() for p in (m.text or "").split(",") if p.strip()]
    if len(parts) != k:
        await m.answer(f"Нужно указать ровно {k} возраст(ов). Попробуйте е
        return
    ages = []
    for p in parts:
        if not re.fullmatch(r"\d{1,2}", p):
            await m.answer("Возраст должен быть числом. Введите ещё раз:",
            return
        ages.append(int(p))
    await state.update_data(kids_ages=ages)
    await push_state(state, ZeroFlow.stars)
    b = stars_inline("zero_stars")
    await m.answer("Звёзды отеля:", reply_markup=KB_GLOBAL)
    await m.answer("Выберите:", reply_markup=b.as_markup())

@router.callback_query(F.data.startswith("zero_stars:"))
async def zero_stars_cb(c: CallbackQuery, state: FSMContext):
    await c.answer()
    val = c.data.split(":", 1)[1]
    if val == "manual":
        await push_state(state, ZeroFlow.stars_manual)
        await c.message.answer("Введите звёзды отеля текстом (например: 4★
        return
    await state.update_data(stars=val)
    await push_state(state, ZeroFlow.dates_mode)
    b = dates_mode_inline("zero_dates")
    await c.message.answer("Даты поездки:", reply_markup=KB_GLOBAL)
    await c.message.answer("Выберите:", reply_markup=b.as_markup())
```

```
@router.message(ZeroFlow.stars_manual)
async def zero_stars_manual(m: Message, state: FSMContext):
    txt = (m.text or "").strip()
    if not txt:
        await m.answer("Введите звёзды отеля:", reply_markup=KB_GLOBAL)
        return
    await state.update_data(stars=txt)
    await push_state(state, ZeroFlow.dates_mode)
    b = dates_mode_inline("zero_dates")
    await m.answer("Даты поездки:", reply_markup=KB_GLOBAL)
    await m.answer("Выберите:", reply_markup=b.as_markup())

@router.callback_query(F.data.startswith("zero_dates:"))
async def zero_dates_cb(c: CallbackQuery, state: FSMContext):
    await c.answer()
    mode = c.data.split(":", 1)[1]
    if mode == "manual":
        await push_state(state, ZeroFlow.dates_manual)
        await c.message.answer("Введите даты текстом (например: 10.01–20.01)", reply_markup=KB_GLOBAL)
        return
    if mode == "exact":
        await push_state(state, ZeroFlow.dates_manual)
        await c.message.answer("Введите точные даты в формате ДД.ММ–ДД.ММ", reply_markup=KB_GLOBAL)
        return
    if mode == "approx":
        b = dates_approx_inline("zero_dates_approx")
        await push_state(state, ZeroFlow.dates_mode) # keep mode state for next step
        await c.message.answer("Выберите примерный период:", reply_markup=b.as_markup())
        await c.message.answer("Период:", reply_markup=b.as_markup())
        return

@router.callback_query(F.data.startswith("zero_dates_approx:"))
async def zero_dates_approx_cb(c: CallbackQuery, state: FSMContext):
    await c.answer()
    val = c.data.split(":", 1)[1]
    if val == "manual":
        await push_state(state, ZeroFlow.dates_manual)
        await c.message.answer("Введите период текстом (например: 'июль 2024–август 2024')", reply_markup=KB_GLOBAL)
        return
    await state.update_data(dates=val)
    await push_state(state, ZeroFlow.budget)
    b = budget_inline("zero_budget")
    await c.message.answer("Бюджет:", reply_markup=KB_GLOBAL)
```

```
        await c.message.answer("Выберите:", reply_markup=b.as_markup())

@router.message(ZeroFlow.dates_manual)
async def zero_dates_manual(m: Message, state: FSMContext):
    txt = (m.text or "").strip()
    if not txt:
        await m.answer("Введите даты/период:", reply_markup=KB_GLOBAL)
        return
    await state.update_data(dates=txt)
    await push_state(state, ZeroFlow.budget)
    b = budget_inline("zero_budget")
    await m.answer("Бюджет:", reply_markup=KB_GLOBAL)
    await m.answer("Выберите:", reply_markup=b.as_markup())

@router.callback_query(F.data.startswith("zero_budget:"))
async def zero_budget_cb(c: CallbackQuery, state: FSMContext):
    await c.answer()
    val = c.data.split(":", 1)[1]
    if val == "manual":
        await push_state(state, ZeroFlow.budget_manual)
        await c.message.answer("Введите бюджет текстом (например: 150 000)")
        return
    await state.update_data(budget=val)
    await push_state(state, ZeroFlow.rest_type)
    b = rest_type_inline("zero_type")
    await c.message.answer("Тип отдыха:", reply_markup=KB_GLOBAL)
    await c.message.answer("Выберите:", reply_markup=b.as_markup())

@router.message(ZeroFlow.budget_manual)
async def zero_budget_manual(m: Message, state: FSMContext):
    txt = (m.text or "").strip()
    if not txt:
        await m.answer("Введите бюджет текстом:", reply_markup=KB_GLOBAL)
        return
    await state.update_data(budget=txt)
    await push_state(state, ZeroFlow.rest_type)
    b = rest_type_inline("zero_type")
    await m.answer("Тип отдыха:", reply_markup=KB_GLOBAL)
    await m.answer("Выберите:", reply_markup=b.as_markup())

@router.callback_query(F.data.startswith("zero_type:"))
async def zero_type_cb(c: CallbackQuery, state: FSMContext):
    await c.answer()
```



```

val = c.data.split(":", 1)[1]
if val == "manual":
    await push_state(state, ZeroFlow.rest_type_manual)
    await c.message.answer("Введите тип отдыха текстом:", reply_markup=
    return
await state.update_data(rest_type=val)
# Choose direction from sheet
await state.update_data(dir_page=0)
await push_state(state, ZeroFlow.choose_direction)
dirs = get_available_directions()
b = paginate_buttons(dirs, 0, 8, "zero_dir")
await c.message.answer("Выберите направление из доступных в базе:", re
await c.message.answer("Направления:", reply_markup=b.as_markup())

@router.message(ZeroFlow.rest_type_manual)
async def zero_type_manual(m: Message, state: FSMContext):
    txt = (m.text or "").strip()
    if not txt:
        await m.answer("Введите тип отдыха текстом:", reply_markup=KB_GLOB
        return
    await state.update_data(rest_type=txt)
    await state.update_data(dir_page=0)
    await push_state(state, ZeroFlow.choose_direction)
    dirs = get_available_directions()
    b = paginate_buttons(dirs, 0, 8, "zero_dir")
    await m.answer("Выберите направление из доступных в базе:", reply_mark
    await m.answer("Направления:", reply_markup=b.as_markup())

@router.callback_query(F.data.startswith("zero_dir:page:"))
async def zero_dir_page(c: CallbackQuery, state: FSMContext):
    await c.answer()
    page = int(c.data.split(":")[-1])
    await state.update_data(dir_page=page)
    dirs = get_available_directions()
    b = paginate_buttons(dirs, page, 8, "zero_dir")
    await c.message.edit_reply_markup(reply_markup=b.as_markup())

@router.callback_query(F.data.startswith("zero_dir:pick:"))
async def zero_dir_pick(c: CallbackQuery, state: FSMContext, bot: Bot):
    await c.answer()
    direction = c.data.split(":", 2)[2]
    await state.update_data(direction=direction)

```

```

tours = get_tours_for_direction(direction)
profile = build_zero_profile_text(await state.get_data())
ranked = rank_tours(profile, tours)
# Store ordered indices
ordered = [i for i, _ in ranked] if ranked else list(range(len(tours)))
await state.update_data(tours=tours, tour_order=ordered, tour_offset=0)

await push_state(state, ZeroFlow.show_tours)
await show_next_tours(bot, c.message.chat.id, state)

async def show_next_tours(bot: Bot, chat_id: int, state: FSMContext):
    data = await state.get_data()
    tours = data.get("tours", [])
    order = data.get("tour_order", [])
    offset = int(data.get("tour_offset", 0))

    if not tours or not order:
        await bot.send_message(chat_id, "В базе нет туров по этому направл")
        await state.update_data(dir_page=0)
        await state.set_state(ZeroFlow.choose_direction)
        dirs = get_available_directions()
        b = paginate_buttons(dirs, 0, 8, "zero_dir")
        await bot.send_message(chat_id, "Направления:", reply_markup=b.as_)
        return

    chunk = order[offset:offset+2]
    if not chunk:
        await bot.send_message(chat_id, "Больше вариантов нет. Можно подтв")
        await push_state(state, ZeroFlow.summary)
        dummy = Message(message_id=0, date=None, chat=None) # not used
        # We'll render summary via separate call in handlers; here just ca
        await render_zero_summary_to_chat(bot, chat_id, state)
        return

    for idx_in_order, tour_idx in enumerate(chunk, start=1):
        t = tours[tour_idx]
        url = t.get("URL", "")
        desc = t.get("описание типа", "") or "Описание скоро будет добавле
        n_global = offset + idx_in_order
        caption = f"Вариант типа #{n_global}\n\n{desc}"
        kb = InlineKeyboardBuilder()
        kb.row(InlineKeyboardButton(text="Выбрать этот тип", callback_data

```

```

# send photo if possible, else send text+url
if url:
    try:
        await bot.send_photo(chat_id, photo=url, caption=caption,
    except Exception:
        await bot.send_message(chat_id, caption + (f"\n\nФото: {ur
else:
    await bot.send_message(chat_id, caption, reply_markup=kb.as_ma

new_offset = offset + len(chunk)
await state.update_data(tour_offset=new_offset)

# show "more" button if there are more
if new_offset < len(order):
    kb_more = InlineKeyboardBuilder()
    kb_more.row(InlineKeyboardButton(text="Показать ещё варианты", cal
    await bot.send_message(chat_id, "Показать ещё варианты?", reply_ma

@router.callback_query(F.data == "zero_more")
async def zero_more(c: CallbackQuery, state: FSMContext, bot: Bot):
    await c.answer()
    await show_next_tours(bot, c.message.chat.id, state)

@router.callback_query(F.data.startswith("zero_tour_pick:"))
async def zero_pick_tour(c: CallbackQuery, state: FSMContext, bot: Bot):
    await c.answer()
    tour_idx = int(c.data.split(":")[1])
    data = await state.get_data()
    tours = data.get("tours", [])
    if not tours or tour_idx >= len(tours):
        await c.message.answer("Не получилось выбрать тур. Попробуйте снов
        return
    chosen = tours[tour_idx]
    await state.update_data(chosen_tour=chosen)
    await push_state(state, ZeroFlow.summary)
    await render_zero_summary_to_chat(bot, c.message.chat.id, state)

async def render_zero_summary_to_chat(bot: Bot, chat_id: int, state: FSMCo
    data = await state.get_data()
    adults = data.get("adults")
    kids = data.get("kids")
    kids_ages = data.get("kids_ages")
    stars = data.get("stars")

```

```

dates = data.get("dates")
budget = data.get("budget")
rest_type = data.get("rest_type")
direction = data.get("direction")
chosen = data.get("chosen_tour")

txt = [
    "📌 Анкета:",
    f"Направление: {direction}",
    f"Состав: взрослые {adults}, дети {kids}" + (f" (возраст: {kids_ag"
    f"Звёзды отеля: {stars}",
    f"Даты: {dates}",
    f"Бюджет: {budget}",
    f"Тип отдыха: {rest_type}",
    "",
]
if chosen:
    txt += [
        "🎯 Подобранный тур:",
        f"Описание: {chosen.get('описание тура', '') or '-'}",
        f"Картинка: {chosen.get('URL', '') or '-'}",
        "",
    ]
txt.append("Подтвердить анкету или изменить?")

kb = InlineKeyboardBuilder()
kb.row(InlineKeyboardButton(text="✅ Подтвердить анкету", callback_dat
kb.row(InlineKeyboardButton(text="✏ Изменить", callback_data="zero_ed
await bot.send_message(chat_id, "\n".join(txt), reply_markup=kb.as_mar

async def render_zero_summary(m: Message, state: FSMContext):
    # kept for back-render hook, uses message.answer
    data = await state.get_data()
    adults = data.get("adults")
    kids = data.get("kids")
    kids_ages = data.get("kids_ages")
    stars = data.get("stars")
    dates = data.get("dates")
    budget = data.get("budget")
    rest_type = data.get("rest_type")
    direction = data.get("direction")
    chosen = data.get("chosen_tour")

```

```

txt = [
    "📌 Анкета:",
    f"Направление: {direction}",
    f"Состав: взрослые {adults}, дети {kids}" + (f" (возраст: {kids_ag"
    f"Звёзды отеля: {stars}",
    f"Даты: {dates}",
    f"Бюджет: {budget}",
    f"Тип отдыха: {rest_type}",
    "",
]
if chosen:
    txt += [
        "🎯 Подобранный тур:",
        f"Описание: {chosen.get('описание тура', '') or '-'}",
        f"Картинка: {chosen.get('URL', '') or '-'}",
        "",
    ]
txt.append("Подтвердить анкету или изменить?")

```

```

kb = InlineKeyboardBuilder()
kb.row(InlineKeyboardButton(text="✅ Подтвердить анкету", callback_data="zero_ed
kb.row(InlineKeyboardButton(text="✏ Изменить", callback_data="zero_ed
await m.answer("\n".join(txt), reply_markup=kb.as_markup())

```

```

@router.callback_query(F.data == "zero_edit")
async def zero_edit(c: CallbackQuery, state: FSMContext):
    await c.answer()
    await push_state(state, ZeroFlow.edit_field)
    kb = InlineKeyboardBuilder()
    kb.row(InlineKeyboardButton(text="Направление", callback_data="zero_ed
    kb.row(InlineKeyboardButton(text="Состав", callback_data="zero_edit_fi
    kb.row(InlineKeyboardButton(text="Звёзды отеля", callback_data="zero_e
    kb.row(InlineKeyboardButton(text="Даты", callback_data="zero_edit_fiel
    kb.row(InlineKeyboardButton(text="Бюджет", callback_data="zero_edit_fi
    kb.row(InlineKeyboardButton(text="Тип отдыха", callback_data="zero_edi
    kb.row(InlineKeyboardButton(text="Тип", callback_data="zero_edit_field
    await c.message.answer("Что изменить?", reply_markup=kb.as_markup())

```

```

@router.callback_query(F.data.startswith("zero_edit_field:"))
async def zero_edit_field_pick(c: CallbackQuery, state: FSMContext, bot: B
    await c.answer()
    field = c.data.split(":")[1]

```

```
if field == "direction":
    await state.update_data(dir_page=0, chosen_tour=None) # reset cho
    await state.set_state(ZeroFlow.choose_direction)
    dirs = get_available_directions()
    b = paginate_buttons(dirs, 0, 8, "zero_dir")
    await c.message.answer("Выберите направление из базы:", reply_mark
    await c.message.answer("Направления:", reply_markup=b.as_markup())
    return

if field == "party":
    # re-ask adults -> kids -> ages; then return to summary
    await state.set_state(ZeroFlow.adults)
    b = adults_inline("zero_adults")
    await c.message.answer("Сколько взрослых?", reply_markup=KB_GLOBAL
    await c.message.answer("Выберите:", reply_markup=b.as_markup())
    return

if field == "stars":
    await state.set_state(ZeroFlow.stars)
    b = stars_inline("zero_stars")
    await c.message.answer("Звёзды отеля:", reply_markup=KB_GLOBAL)
    await c.message.answer("Выберите:", reply_markup=b.as_markup())
    return

if field == "dates":
    await state.set_state(ZeroFlow.dates_mode)
    b = dates_mode_inline("zero_dates")
    await c.message.answer("Даты поездки:", reply_markup=KB_GLOBAL)
    await c.message.answer("Выберите:", reply_markup=b.as_markup())
    return

if field == "budget":
    await state.set_state(ZeroFlow.budget)
    b = budget_inline("zero_budget")
    await c.message.answer("Бюджет:", reply_markup=KB_GLOBAL)
    await c.message.answer("Выберите:", reply_markup=b.as_markup())
    return

if field == "rest_type":
    await state.set_state(ZeroFlow.rest_type)
    b = rest_type_inline("zero_type")
    await c.message.answer("Тип отдыха:", reply_markup=KB_GLOBAL)
    await c.message.answer("Выберите:", reply_markup=b.as_markup())
```

```

        return

    if field == "tour":
        # rerun ranking with current direction
        data = await state.get_data()
        direction = data.get("direction")
        tours = get_tours_for_direction(direction)
        profile = build_zero_profile_text(data)
        ranked = rank_tours(profile, tours)
        ordered = [i for i, _ in ranked] if ranked else list(range(len(tou
        await state.update_data(tours=tours, tour_order=ordered, tour_offs
        await state.set_state(ZeroFlow.show_tours)
        await show_next_tours(bot, c.message.chat.id, state)
        return

@router.callback_query(F.data == "zero_confirm")
async def zero_confirm(c: CallbackQuery, state: FSMContext):
    await c.answer()
    await push_state(state, ZeroFlow.contact_phone)
    await c.message.answer("Отправьте номер телефона (кнопкой) или напишит

@router.message(ZeroFlow.contact_phone)
async def zero_phone(m: Message, state: FSMContext):
    phone = None
    if m.contact and m.contact.phone_number:
        phone = normalize_phone(m.contact.phone_number)
    else:
        phone = normalize_phone(m.text or "")
    if not phone:
        await m.answer("Не распознал телефон. Отправьте кнопкой или напиши
        return
    await state.update_data(phone=phone)
    await push_state(state, ZeroFlow.contact_name)
    await m.answer("Введите Фамилию Имя:", reply_markup=KB_GLOBAL)

@router.message(ZeroFlow.contact_name)
async def zero_name(m: Message, state: FSMContext):
    fio = re.sub(r"\s+", " ", (m.text or "").strip())
    if len(fio.split()) < 2:
        await m.answer("Нужно в формате «Фамилия Имя». Введите ещё раз:",
        return
    data = await state.get_data()
    phone = data.get("phone", "")

```

```

    await m.answer(
        f"Спасибо, {fio}! Мы отправили вашу заявку менеджеру – скоро с вам
        reply_markup=KB_START
    )
    await reset_to_start(state)

# -----
# 16) TTS callbacks (IDEA branch only)
# -----
@router.callback_query(F.data.startswith("tts:"))
async def tts_cb(c: CallbackQuery, bot: Bot):
    await c.answer()
    key = c.data.split(":", 1)[1]
    payload = TTS_STORE.get(key)
    if not payload:
        await c.message.answer("Не нашёл текст для озвучки (сообщение слиш
        return
    text = payload.get("text", "")
    try:
        mp3 = await openai_tts_mp3_bytes(text)
        from aiogram.types import BufferedInputFile
        audio = BufferedInputFile(mp3, filename="tts.mp3")
        await bot.send_audio(c.message.chat.id, audio=audio, caption="🔊 0
    except Exception as e:
        await c.message.answer(f"Ошибка озвучки: {e}")

# -----
# 17) Fallback: if user types random text outside flow
# -----
@router.message()
async def fallback(m: Message, state: FSMContext):
    st = await state.get_state()
    if not st:
        await m.answer("Нажмите /start", reply_markup=KB_START)
        return
    # If in consultation but user typed something odd handled above
    await m.answer("Пожалуйста, используйте кнопки или ответьте на вопрос.

# -----
# 18) Run bot
# -----
async def main():
    bot = Bot(TELEGRAM_BOT_TOKEN)

```



```

dp = Dispatcher(storage=MemoryStorage())
dp.include_router(router)

# Commands menu
try:
    from aiogram.types import BotCommand
    await bot.set_my_commands([
        BotCommand(command="start", description="Начать заново"),
        BotCommand(command="help", description="Помощь"),
        BotCommand(command="contacts", description="Контакты менеджера"),
        BotCommand(command="site", description="Сайт"),
    ])
except Exception:
    pass

# Avoid webhook conflicts
try:
    await bot.delete_webhook(drop_pending_updates=True)
except Exception:
    pass

await dp.start_polling(bot, allowed_updates=dp.resolve_used_update_type)

asyncio.run(main())

```



Сейчас есть несколько ошибок, исправь их

1. в ветке "Есть идея", после ответа на вопрос выводит на главное меню у меня есть идея и подобрать с нуля, нет озвучки сообщения
2. в ветке "Подобрать с нуля" после того как нажимаю выбрать тип отдых не выводит тур и вообще ничего не выводит что-то ломается вот такую ошибку выдает

File "/usr/local/lib/python3.12/dist-packages/aiogram/dispatcher/event/handler.py", line 43, in call

```

    return await wrapped()
    ^^^^^^^^^^^^^^^^^^^

```

File "/tmp/ipython-input-2787776957.py", line 1086, in


```
import tempfile
import random
from typing import Any, Dict, List, Optional, Tuple

import pandas as pd
import requests
import nest_asyncio

from aiogram import Bot, Dispatcher, Router, F
from aiogram.types import (
    Message,
    CallbackQuery,
    ReplyKeyboardMarkup,
    KeyboardButton,
    InlineKeyboardButton,
)
from aiogram.utils.keyboard import InlineKeyboardBuilder
from aiogram.filters import Command, CommandStart
from aiogram.fsm.storage.memory import MemoryStorage
from aiogram.fsm.state import State, StatesGroup
from aiogram.fsm.context import FSMContext

from openai import OpenAI

nest_asyncio.apply()

# -----
# 1) Secrets (ONLY Colab)
# -----
try:
    from google.colab import userdata # type: ignore
except Exception as e:
    raise RuntimeError("Этот код рассчитан на Google Colab (нужен google.c

TELEGRAM_BOT_TOKEN = userdata.get("TELEGRAM_BOT_TOKEN")
OPENAI_API_KEY = userdata.get("OPENAI_API_KEY")

if not TELEGRAM_BOT_TOKEN:
    raise ValueError("Не найден секрет TELEGRAM_BOT_TOKEN в Colab Secrets.
if not OPENAI_API_KEY:
    raise ValueError("Не найден секрет OPENAI_API_KEY в Colab Secrets.")

# -----
```

```

# 2) Config
# -----
SHEET_URL = "https://docs.google.com/spreadsheets/d/1aypI3sQSexD6MP4kawiRN
SHEET_ID = re.search(r"/spreadsheets/d/([a-zA-Z0-9_-]+)", SHEET_URL).group
SHEET_CSV_URL = f"https://docs.google.com/spreadsheets/d/{SHEET_ID}/gviz/t

OPENAI_TTS_MODEL = "gpt-4o-mini-tts"
OPENAI_TTS_VOICE = "coral"
OPENAI_CHAT_MODEL = "gpt-5"

oai = OpenAI(api_key=OPENAI_API_KEY)

# -----
# 3) Google Sheets helpers
# -----
_SHEET_CACHE: Dict[str, Any] = {"ts": 0, "df": None}

def _norm_col(s: str) -> str:
    return re.sub(r"\s+", " ", str(s)).strip().lower()

def _extract_drive_file_id(url: str) -> Optional[str]:
    if not url:
        return None
    m = re.search(r"/d/([a-zA-Z0-9_-]+)", url)
    if m:
        return m.group(1)
    m = re.search(r"[?&]id=([a-zA-Z0-9_-]+)", url)
    if m:
        return m.group(1)
    return None

def _to_direct_url(url: str) -> str:
    if not url:
        return url
    fid = _extract_drive_file_id(url)
    if fid:
        return f"https://drive.google.com/uc?export=download&id={fid}"
    return url

def load_sheet_df(force: bool = False, ttl_sec: int = 300) -> pd.DataFrame
    now = time.time()
    if not force and _SHEET_CACHE["df"] is not None and (now - _SHEET_CACH
        return _SHEET_CACHE["df"]

```

```

resp = requests.get(SHEET_CSV_URL, timeout=25)
resp.raise_for_status()
df = pd.read_csv(pd.io.common.StringIO(resp.text))

cols = {_norm_col(c): c for c in df.columns}
country_col = cols.get("страна") or next((cols[k] for k in cols if "ст
url_col = cols.get("url") or next((cols[k] for k in cols if k == "url"
desc_col = cols.get("описание типа") or next((cols[k] for k in cols if

if not country_col or not url_col:
    raise ValueError(
        "В таблице не найдены обязательные колонки 'страна' и 'URL'. "
        f"Найденные колонки: {list(df.columns)}"
    )

df = df.rename(columns={country_col: "страна", url_col: "URL"})
if desc_col:
    df = df.rename(columns={desc_col: "описание типа"})
else:
    df["описание типа"] = ""

df["страна"] = df["страна"].astype(str).fillna("").str.strip()
df["URL"] = df["URL"].astype(str).fillna("").str.strip()
df["описание типа"] = df["описание типа"].astype(str).fillna("").str.s

_SHEET_CACHE["ts"] = now
_SHEET_CACHE["df"] = df
return df

def base_country_name(s: str) -> str:
    s = re.sub(r"\s+", " ", str(s)).strip()
    s = re.sub(r"\s+\d+$", "", s).strip()
    return s

def get_available_directions() -> List[str]:
    df = load_sheet_df()
    bases = sorted({base_country_name(x) for x in df["страна"].tolist() if
    return bases

def get_tours_for_direction(direction_base: str) -> List[Dict[str, str]]:
    df = load_sheet_df()
    rows = df.to_dict(orient="records")

```

```

tours: List[Dict[str, str]] = []
for r in rows:
    c = base_country_name(r.get("страна", ""))
    if c.lower() == direction_base.lower():
        tours.append({
            "страна": str(r.get("страна", "")).strip(),
            "URL": _to_direct_url(str(r.get("URL", "")).strip()),
            "описание типа": str(r.get("описание типа", "")).strip(),
        })
return tours

# -----
# 4) Similarity (simple)
# -----
_RU_WORD = re.compile(r"[a-zA-Za-яA-ЯёЁ0-9]+")

def tokenize(text: str) -> List[str]:
    return _RU_WORD.findall((text or "").lower())

def jaccard_score(a: str, b: str) -> float:
    A = set(tokenize(a))
    B = set(tokenize(b))
    if not A or not B:
        return 0.0
    return len(A & B) / (len(A | B) + 1e-9)

def build_zero_profile_text(data: Dict[str, Any]) -> str:
    parts = []
    parts.append(f"взрослые: {data.get('adults')}")
    parts.append(f"дети: {data.get('kids')}")
    if data.get("kids_ages"):
        parts.append(f"возраст детей: {data.get('kids_ages')}")
    parts.append(f"звезды: {data.get('stars')}")
    parts.append(f"даты: {data.get('dates')}")
    parts.append(f"бюджет: {data.get('budget')}")
    parts.append(f"тип отдыха: {data.get('rest_type')}")
    return ", ".join([p for p in parts if p and "None" not in str(p)])

def rank_tours(profile_text: str, tours: List[Dict[str, str]]) -> List[Tuple]:
    scored: List[Tuple[int, float]] = []
    for i, t in enumerate(tours):
        score = jaccard_score(profile_text, t.get("описание типа", "") or
        scored.append((i, score))

```

```

scored.sort(key=lambda x: x[1], reverse=True)
return scored

# -----
# 5) OpenAI helpers (TTS + web_search)
# -----
async def openai_tts_mp3_bytes(text: str) -> bytes:
    text = (text or "").strip()[:4096] or " "
    with tempfile.NamedTemporaryFile(suffix=".mp3", delete=False) as tmp:
        tmp_path = tmp.name
    try:
        with oai.audio.speech.with_streaming_response.create(
            model=OPENAI_TTS_MODEL,
            voice=OPENAI_TTS_VOICE,
            input=text,
            instructions="Говори чётко, дружелюбно. Это голос ИИ."
        ) as response:
            response.stream_to_file(tmp_path)
        with open(tmp_path, "rb") as f:
            return f.read()
    finally:
        try:
            os.remove(tmp_path)
        except Exception:
            pass

def looks_like_travel_question(text: str) -> bool:
    t = (text or "").lower()
    if len(t) < 3:
        return False
    kw = [
        "виза", "паспорт", "перелет", "самолет", "аэропорт", "багаж", "погода", "
        "звезды", "страховк", "прививк", "маршрут", "что посмотреть", "достопри
        "бюджет", "трансфер", "экскурс", "тур", "поездк", "пляж", "горы", "сафари
        "безопасн", "сим-карт", "связь", "интернет", "розетка", "время", "еда", "
    ]
    return any(k in t for k in kw) or "?" in t

async def openai_answer_with_web_search(profile: str, question: str) -> st
    prompt = (
        "Ты – туристический консультант. Отвечай кратко, практично, по дел
        "Если данных недостаточно – задай 1 уточняющий вопрос в конце.\n\n
        f"Анкета клиента: {profile}\n"

```

```

f"Вопрос клиента: {question}\n\n"
"Используй web_search только для актуальной информации. "
"Не придумывай факты. Если есть ссылки/источники — оставь их в отв
)
resp = oai.responses.create(
    model=OPENAI_CHAT_MODEL,
    tools=[{"type": "web_search"}],
    input=prompt,
    reasoning={"effort": "low"},
)
return (resp.output_text or "").strip() or "Не смог сформировать ответ

# -----
# 6) Keyboards
# -----
KB_START = ReplyKeyboardMarkup(
    keyboard=[
        [KeyboardButton(text="У меня есть идея")],
        [KeyboardButton(text="Подобрать с нуля")],
    ],
    resize_keyboard=True,
    selective=True,
)

KB_GLOBAL = ReplyKeyboardMarkup(
    keyboard=[KeyboardButton(text="Назад"), KeyboardButton(text="Вернутьс
    resize_keyboard=True,
    selective=True,
)

KB_CONSULT = ReplyKeyboardMarkup(
    keyboard=[
        [KeyboardButton(text="Нет вопросов")],
        [KeyboardButton(text="Назад"), KeyboardButton(text="Вернуться в на
    ],
    resize_keyboard=True,
    selective=True,
)

def kb_contact_request() -> ReplyKeyboardMarkup:
    return ReplyKeyboardMarkup(
        keyboard=[
            [KeyboardButton(text="📞 Отправить номер", request_contact=Tru

```



```
        [KeyboardButton(text="Назад"), KeyboardButton(text="Вернуться")],
        resize_keyboard=True,
        selective=True,
    )

# -----
# 7) FSM States
# -----

class IdeaFlow(StatesGroup):
    direction = State()
    adults = State()
    adults_manual = State()
    kids = State()
    kids_manual = State()
    kids_ages = State()
    budget = State()
    budget_manual = State()
    rest_type = State()
    rest_type_manual = State()
    consult = State()
    contact_phone = State()
    contact_name = State()

class ZeroFlow(StatesGroup):
    adults = State()
    adults_manual = State()
    kids = State()
    kids_manual = State()
    kids_ages = State()
    stars = State()
    stars_manual = State()
    dates_mode = State()
    dates_manual = State()
    budget = State()
    budget_manual = State()
    rest_type = State()
    rest_type_manual = State()
    choose_direction = State()
    show_tours = State()
    summary = State()
    edit_field = State()
    contact_phone = State()
```

```

        contact_name = State()

# -----
# 8) Back stack helpers
# -----
async def push_state(ctx: FSMContext, state: State):
    data = await ctx.get_data()
    hist = data.get("history", [])
    st = await ctx.get_state()
    if st:
        hist.append(st)
    await ctx.update_data(history=hist)
    await ctx.set_state(state)

async def go_back(ctx: FSMContext) -> Optional[str]:
    data = await ctx.get_data()
    hist = data.get("history", [])
    if not hist:
        return None
    prev = hist.pop()
    await ctx.update_data(history=hist)
    await ctx.set_state(prev)
    return prev

async def reset_to_start(ctx: FSMContext):
    await ctx.clear()

# -----
# 9) TTS Store (idea branch)
# -----
TTS_STORE: Dict[str, Dict[str, Any]] = {}
_TTS_COUNTER = 0

def make_tts_key(chat_id: int) -> str:
    global _TTS_COUNTER
    _TTS_COUNTER += 1
    # short key <= 64 chars
    return f"{chat_id}:{int(time.time()*1000)}:{_TTS_COUNTER}:{random.rand

def add_tts_button(builder: InlineKeyboardBuilder, tts_key: str) -> Inline
    builder.row(InlineKeyboardButton(text="🔊 Озвучить", callback_data=f"t
    return builder

```

```

async def send_idea_msg(
    bot: Bot,
    chat_id: int,
    text: str,
    reply_kb: Optional[ReplyKeyboardMarkup] = None,
    inline_builder: Optional[InlineKeyboardBuilder] = None,
):
    # message with inline (tts always), then optionally message that sets
    b = inline_builder or InlineKeyboardBuilder()
    key = make_tts_key(chat_id)
    TTS_STORE[key] = {"chat_id": chat_id, "text": text}
    add_tts_button(b, key)
    await bot.send_message(chat_id, text, reply_markup=b.as_markup())
    if reply_kb is not None:
        # IMPORTANT: not blank -> Telegram reliably shows reply keyboard
        await bot.send_message(chat_id, "Выберите действие:", reply_markup=

# -----
# 10) Inline keyboards builders
# -----
def adults_inline(prefix: str) -> InlineKeyboardBuilder:
    b = InlineKeyboardBuilder()
    for n in [1,2,3,4,5,6,7,8]:
        b.button(text=str(n), callback_data=f"{prefix}:{n}")
    b.adjust(4,4)
    b.row(InlineKeyboardButton(text="Ввести вручную", callback_data=f"{pre
    return b

def kids_inline(prefix: str) -> InlineKeyboardBuilder:
    b = InlineKeyboardBuilder()
    for n in [0,1,2,3,4,5]:
        b.button(text=str(n), callback_data=f"{prefix}:{n}")
    b.adjust(3,3)
    b.row(InlineKeyboardButton(text="Ввести вручную", callback_data=f"{pre
    return b

def budget_inline(prefix: str) -> InlineKeyboardBuilder:
    b = InlineKeyboardBuilder()
    b.button(text="До 50к", callback_data=f"{prefix}:До 50к")
    b.button(text="50-100к", callback_data=f"{prefix}:50-100к")
    b.button(text="100-200к", callback_data=f"{prefix}:100-200к")
    b.button(text="200к+", callback_data=f"{prefix}:200к+")
    b.adjust(2,2)

```

```

b.row(InlineKeyboardButton(text="Ввести вручную", callback_data=f"{pre
return b

def rest_type_inline(prefix: str) -> InlineKeyboardBuilder:
    b = InlineKeyboardBuilder()
    opts = ["Пляж/релакс", "Экскурсии", "Активный", "Семейный", "Природа/горы"]
    for o in opts:
        b.button(text=o, callback_data=f"{prefix}:{o}")
    b.adjust(2,2,2)
    b.row(InlineKeyboardButton(text="Ввести вручную", callback_data=f"{pre
return b

def stars_inline(prefix: str) -> InlineKeyboardBuilder:
    b = InlineKeyboardBuilder()
    for o in ["Любые", "3★", "4★", "5★"]:
        b.button(text=o, callback_data=f"{prefix}:{o}")
    b.adjust(2,2)
    b.row(InlineKeyboardButton(text="Ввести вручную", callback_data=f"{pre
return b

def dates_mode_inline(prefix: str) -> InlineKeyboardBuilder:
    b = InlineKeyboardBuilder()
    b.button(text="Точные", callback_data=f"{prefix}:exact")
    b.button(text="Примерно", callback_data=f"{prefix}:approx")
    b.adjust(2)
    b.row(InlineKeyboardButton(text="Ввести вручную", callback_data=f"{pre
return b

def dates_approx_inline(prefix: str) -> InlineKeyboardBuilder:
    b = InlineKeyboardBuilder()
    opts = ["В этом месяце", "В следующем месяце", "Через 2–3 месяца", "Лето"]
    for o in opts:
        b.button(text=o, callback_data=f"{prefix}:{o}")
    b.adjust(2,2,3)
    b.row(InlineKeyboardButton(text="Ввести вручную", callback_data=f"{pre
return b

def paginate_buttons(items: List[str], page: int, per_page: int, cb_prefix
# FIXED: no generator indexing; build nav row explicitly
b = InlineKeyboardBuilder()
start = page * per_page
chunk = items[start:start+per_page]
for it in chunk:

```

```

        b.button(text=it, callback_data=f"{cb_prefix}:pick:{it}")
    b.adjust(2,2,2,2)

nav_buttons: List[InlineKeyboardButton] = []
if page > 0:
    nav_buttons.append(InlineKeyboardButton(text="⬅", callback_data=f
if start + per_page < len(items):
    nav_buttons.append(InlineKeyboardButton(text="➡", callback_data=f
if nav_buttons:
    b.row(*nav_buttons)
return b

# -----
# 11) Utils
# -----
def normalize_phone(s: str) -> Optional[str]:
    digits = re.sub(r"\D+", "", s or "")
    if len(digits) < 10:
        return None
    return digits

def safe_text(s: Any) -> str:
    return re.sub(r"\s+", " ", str(s or "")).strip()

# -----
# 12) Router + commands
# -----
router = Router()

@router.message(Command("help"))
async def cmd_help(m: Message):
    await m.answer(
        "Я AI-тип-агент: помогу собрать заявку, проконсультировать по напр
        "Команды:\n"
        "/start — начать заново\n"
        "/help — помощь\n"
        "/contacts — контакты менеджера\n"
        "/site — сайт",
        reply_markup=KB_START
    )

@router.message(Command("contacts"))
async def cmd_contacts(m: Message):

```

```

    await m.answer("Менеджер: Алёна\nТелефон: 89998765432")

@router.message(Command("site"))
async def cmd_site(m: Message):
    await m.answer("https://neural-university.ru/")

@router.message(CommandStart())
async def start(m: Message, state: FSMContext):
    await reset_to_start(state)
    await m.answer(
        "Привет! Я AI-тур-агент – помогу собрать заявку. У вас уже есть ид
        reply_markup=KB_START
    )

@router.message(F.text == "Вернуться в начало")
async def go_home(m: Message, state: FSMContext):
    await reset_to_start(state)
    await m.answer(
        "Привет! Я AI-тур-агент – помогу собрать заявку. У вас уже есть ид
        reply_markup=KB_START
    )

@router.message(F.text == "Назад")
async def back(m: Message, state: FSMContext, bot: Bot):
    prev = await go_back(state)
    if not prev:
        await m.answer("Вы уже в начале. Нажмите «Вернуться в начало».", r
        return

    st = await state.get_state()
    chat_id = m.chat.id

    # IDEA re-render
    if st == IdeaFlow.direction.state:
        await send_idea_msg(bot, chat_id, "Куда хотите поехать? Напишите н
    elif st == IdeaFlow.adults.state:
        await send_idea_msg(bot, chat_id, "Сколько взрослых?", reply_kb=KB
    elif st == IdeaFlow.kids.state:
        await send_idea_msg(bot, chat_id, "Сколько детей?", reply_kb=KB_GL
    elif st == IdeaFlow.kids_ages.state:
        data = await state.get_data()
        n = int(data.get("kids", 0))
        await send_idea_msg(bot, chat_id, f"Укажите возраст каждого ребёнк

```

```
elif st == IdeaFlow.budget.state:
    await send_idea_msg(bot, chat_id, "Бюджет:", reply_kb=KB_GLOBAL, i
elif st == IdeaFlow.rest_type.state:
    await send_idea_msg(bot, chat_id, "Тип отдыха:", reply_kb=KB_GLOBA
elif st == IdeaFlow.consult.state:
    await send_idea_msg(bot, chat_id, "Режим консультации включён. Зад

# ZERO re-render (reply keyboard only)
elif st == ZeroFlow.adults.state:
    await m.answer("Сколько взрослых?", reply_markup=KB_GLOBAL)
    await m.answer("Выберите:", reply_markup=adults_inline("zero_adult
elif st == ZeroFlow.kids.state:
    await m.answer("Сколько детей?", reply_markup=KB_GLOBAL)
    await m.answer("Выберите:", reply_markup=kids_inline("zero_kids").
elif st == ZeroFlow.kids_ages.state:
    data = await state.get_data()
    n = int(data.get("kids", 0))
    await m.answer(f"Укажите возраст каждого ребёнка через запятую (вс
elif st == ZeroFlow.stars.state:
    await m.answer("Звёзды отеля:", reply_markup=KB_GLOBAL)
    await m.answer("Выберите:", reply_markup=stars_inline("zero_stars"
elif st == ZeroFlow.dates_mode.state:
    await m.answer("Даты поездки:", reply_markup=KB_GLOBAL)
    await m.answer("Выберите:", reply_markup=dates_mode_inline("zero_d
elif st == ZeroFlow.budget.state:
    await m.answer("Бюджет:", reply_markup=KB_GLOBAL)
    await m.answer("Выберите:", reply_markup=budget_inline("zero_budge
elif st == ZeroFlow.rest_type.state:
    await m.answer("Тип отдыха:", reply_markup=KB_GLOBAL)
    await m.answer("Выберите:", reply_markup=rest_type_inline("zero_ty
elif st == ZeroFlow.choose_direction.state:
    data = await state.get_data()
    page = int(data.get("dir_page", 0))
    dirs = get_available_directions()
    await m.answer("Выберите направление из базы:", reply_markup=KB_GL
    await m.answer("Направления:", reply_markup=paginate_buttons(dirs,
elif st == ZeroFlow.contact_phone.state:
    await m.answer("Отправьте номер телефона (кнопкой) или напишите вр
elif st == IdeaFlow.contact_phone.state:
    await m.answer("Отправьте номер телефона (кнопкой) или напишите вр
else:
    await m.answer("Ок.", reply_markup=KB_GLOBAL)
```

```

# -----
# 13) Branch selection
# -----
@router.message(F.text == "У меня есть идея")
async def choose_idea(m: Message, state: FSMContext, bot: Bot):
    await reset_to_start(state)
    await state.update_data(flow="idea", history=[])
    await push_state(state, IdeaFlow.direction)
    await send_idea_msg(bot, m.chat.id, "Куда хотите поехать? Напишите нап

@router.message(F.text == "Подобрать с нуля")
async def choose_zero(m: Message, state: FSMContext):
    await reset_to_start(state)
    await state.update_data(flow="zero", history=[])
    await push_state(state, ZeroFlow.adults)
    await m.answer("Сколько взрослых?", reply_markup=KB_GLOBAL)
    await m.answer("Выберите:", reply_markup=adults_inline("zero_adults").

# -----
# 14) IDEA flow
# -----
@router.message(IdeaFlow.direction)
async def idea_direction(m: Message, state: FSMContext, bot: Bot):
    txt = safe_text(m.text)
    if not txt:
        await send_idea_msg(bot, m.chat.id, "Напишите направление текстом
        return
    await state.update_data(direction=txt)
    await push_state(state, IdeaFlow.adults)
    await send_idea_msg(bot, m.chat.id, "Сколько взрослых?", reply_kb=KB_G

@router.callback_query(F.data.startswith("idea_adults:"))
async def idea_adults_cb(c: CallbackQuery, state: FSMContext, bot: Bot):
    await c.answer()
    val = c.data.split(":", 1)[1]
    if val == "manual":
        await push_state(state, IdeaFlow.adults_manual)
        await send_idea_msg(bot, c.message.chat.id, "Введите число взрослые
        return
    n = int(val)
    if not (1 <= n <= 8):
        await send_idea_msg(bot, c.message.chat.id, "Взрослые должны быть
        return

```



```
await state.update_data(adults=n)
await push_state(state, IdeaFlow.kids)
await send_idea_msg(bot, c.message.chat.id, "Сколько детей?", reply_kb
```

```
@router.message(IdeaFlow.adults_manual)
```

```
async def idea_adults_manual(m: Message, state: FSMContext, bot: Bot):
    txt = safe_text(m.text)
    if not txt.isdigit():
        await send_idea_msg(bot, m.chat.id, "Нужно число 1–8. Введите ещё")
        return
    n = int(txt)
    if not (1 <= n <= 8):
        await send_idea_msg(bot, m.chat.id, "Взрослые строго 1–8. Введите")
        return
    await state.update_data(adults=n)
    await push_state(state, IdeaFlow.kids)
    await send_idea_msg(bot, m.chat.id, "Сколько детей?", reply_kb=KB_GLOB
```

```
@router.callback_query(F.data.startswith("idea_kids:"))
```

```
async def idea_kids_cb(c: CallbackQuery, state: FSMContext, bot: Bot):
    await c.answer()
    val = c.data.split(":", 1)[1]
    if val == "manual":
        await push_state(state, IdeaFlow.kids_manual)
        await send_idea_msg(bot, c.message.chat.id, "Введите число детей (")
        return
    k = int(val)
    if not (0 <= k <= 5):
        await send_idea_msg(bot, c.message.chat.id, "Дети должны быть 0–5.")
        return
    await state.update_data(kids=k)
    if k > 0:
        await push_state(state, IdeaFlow.kids_ages)
        await send_idea_msg(bot, c.message.chat.id, f"Укажите возраст кажд
    else:
        await push_state(state, IdeaFlow.budget)
        await send_idea_msg(bot, c.message.chat.id, "Бюджет:", reply_kb=KB
```

```
@router.message(IdeaFlow.kids_manual)
```

```
async def idea_kids_manual(m: Message, state: FSMContext, bot: Bot):
    txt = safe_text(m.text)
    if not txt.isdigit():
        await send_idea_msg(bot, m.chat.id, "Нужно число 0–5. Введите ещё
```

```

        return
    k = int(txt)
    if not (0 <= k <= 5):
        await send_idea_msg(bot, m.chat.id, "Дети строго 0–5. Введите ещё")
        return
    await state.update_data(kids=k)
    if k > 0:
        await push_state(state, IdeaFlow.kids_ages)
        await send_idea_msg(bot, m.chat.id, f"Укажите возраст каждого ребёнка")
    else:
        await push_state(state, IdeaFlow.budget)
        await send_idea_msg(bot, m.chat.id, "Бюджет:", reply_kb=KB_GLOBAL,

@router.message(IdeaFlow.kids_ages)
async def idea_kids_ages(m: Message, state: FSMContext, bot: Bot):
    data = await state.get_data()
    k = int(data.get("kids", 0))
    parts = [p.strip() for p in (m.text or "").split(",") if p.strip()]
    if len(parts) != k:
        await send_idea_msg(bot, m.chat.id, f"Нужно указать ровно {k} возр")
        return
    ages = []
    for p in parts:
        if not re.fullmatch(r"\d{1,2}", p):
            await send_idea_msg(bot, m.chat.id, "Возраст должен быть число")
            return
        ages.append(int(p))
    await state.update_data(kids_ages=ages)
    await push_state(state, IdeaFlow.budget)
    await send_idea_msg(bot, m.chat.id, "Бюджет:", reply_kb=KB_GLOBAL, inl

@router.callback_query(F.data.startswith("idea_budget:"))
async def idea_budget_cb(c: CallbackQuery, state: FSMContext, bot: Bot):
    await c.answer()
    val = c.data.split(":", 1)[1]
    if val == "manual":
        await push_state(state, IdeaFlow.budget_manual)
        await send_idea_msg(bot, c.message.chat.id, "Введите бюджет текстом")
        return
    await state.update_data(budget=val)
    await push_state(state, IdeaFlow.rest_type)
    await send_idea_msg(bot, c.message.chat.id, "Тип отдыха:", reply_kb=KB

```

```
@router.message(IdeaFlow.budget_manual)
async def idea_budget_manual(m: Message, state: FSMContext, bot: Bot):
    txt = safe_text(m.text)
    if not txt:
        await send_idea_msg(bot, m.chat.id, "Введите бюджет текстом:", rep
        return
    await state.update_data(budget=txt)
    await push_state(state, IdeaFlow.rest_type)
    await send_idea_msg(bot, m.chat.id, "Тип отдыха:", reply_kb=KB_GLOBAL,

@router.callback_query(F.data.startswith("idea_type:"))
async def idea_type_cb(c: CallbackQuery, state: FSMContext, bot: Bot):
    await c.answer()
    val = c.data.split(":", 1)[1]
    if val == "manual":
        await push_state(state, IdeaFlow.rest_type_manual)
        await send_idea_msg(bot, c.message.chat.id, "Введите тип отдыха те
        return
    await state.update_data(rest_type=val)
    await push_state(state, IdeaFlow.consult)
    await send_idea_msg(
        bot,
        c.message.chat.id,
        "Режим консультации включён. Задайте вопрос по поездке или нажмите
        reply_kb=KB_CONSULT
    )

@router.message(IdeaFlow.rest_type_manual)
async def idea_type_manual(m: Message, state: FSMContext, bot: Bot):
    txt = safe_text(m.text)
    if not txt:
        await send_idea_msg(bot, m.chat.id, "Введите тип отдыха текстом:",
        return
    await state.update_data(rest_type=txt)
    await push_state(state, IdeaFlow.consult)
    await send_idea_msg(
        bot,
        m.chat.id,
        "Режим консультации включён. Задайте вопрос по поездке или нажмите
        reply_kb=KB_CONSULT
    )

@router.message(IdeaFlow.consult, F.text == "Нет вопросов")
```

```

async def idea_no_questions(m: Message, state: FSMContext, bot: Bot):
    data = await state.get_data()
    ank = [
        "📌 Анкета:",
        f"Направление: {data.get('direction')}",
        f"Состав: взрослые {data.get('adults')}, дети {data.get('kids')}"
        f"Бюджет: {data.get('budget')}",
        f"Тип отдыха: {data.get('rest_type')}",
        "",
        "Отправьте номер телефона (кнопкой) или напишите вручную:"
    ]
    await push_state(state, IdeaFlow.contact_phone)
    await send_idea_msg(bot, m.chat.id, "\n".join(ank), reply_kb=kb Contac

```

```
@router.message(IdeaFlow.consult)
```

```

async def idea_consult_question(m: Message, state: FSMContext, bot: Bot):
    txt = safe_text(m.text)
    if txt in {"Назад", "Вернуться в начало"}:
        return
    if not txt:
        await send_idea_msg(bot, m.chat.id, "Напишите вопрос текстом или н
        return

    if not looks_like_travel_question(txt):
        await send_idea_msg(bot, m.chat.id, "Этот вопрос выходит за предел
        return

    data = await state.get_data()
    profile = (
        f"направление: {data.get('direction')}, взрослые: {data.get('adult
        f"бюджет: {data.get('budget')}, тип отдыха: {data.get('rest_type'
    )
    try:
        answer = await openai_answer_with_web_search(profile, txt)
    except Exception as e:
        answer = f"Не получилось выполнить поиск/ответ: {e}"

    # IMPORTANT: keep consult reply keyboard (so it doesn't show start)
    await send_idea_msg(bot, m.chat.id, answer, reply_kb=KB_CONSULT)

```

```
@router.message(IdeaFlow.contact_phone)
```

```

async def idea_phone(m: Message, state: FSMContext, bot: Bot):
    phone = None

```

```

    if m.contact and m.contact.phone_number:
        phone = normalize_phone(m.contact.phone_number)
    else:
        phone = normalize_phone(m.text or "")
    if not phone:
        await send_idea_msg(bot, m.chat.id, "Не распознал телефон. Отправь
        return
    await state.update_data(phone=phone)
    await push_state(state, IdeaFlow.contact_name)
    await send_idea_msg(bot, m.chat.id, "Введите Фамилию Имя:", reply_kb=K

@router.message(IdeaFlow.contact_name)
async def idea_name(m: Message, state: FSMContext, bot: Bot):
    fio = safe_text(m.text)
    if len(fio.split()) < 2:
        await send_idea_msg(bot, m.chat.id, "Нужно в формате «Фамилия Имя»
        return
    data = await state.get_data()
    phone = data.get("phone", "")
    await send_idea_msg(
        bot,
        m.chat.id,
        f"Спасибо, {fio}! Мы отправили вашу заявку менеджеру – скоро с вами
        reply_kb=KB_START
    )
    await reset_to_start(state)

# -----
# 15) ZERO flow
# -----
@router.callback_query(F.data.startswith("zero_adults:"))
async def zero_adults_cb(c: CallbackQuery, state: FSMContext):
    await c.answer()
    val = c.data.split(":", 1)[1]
    if val == "manual":
        await push_state(state, ZeroFlow.adults_manual)
        await c.message.answer("Введите число взрослых (1–8):", reply_mark
        return
    n = int(val)
    if not (1 <= n <= 8):
        await c.message.answer("Взрослые строго 1–8. Выберите снова.", rep
        await c.message.answer("Выберите:", reply_markup=adults_inline("ze
        return

```

```

await state.update_data(adults=n)
await push_state(state, ZeroFlow.kids)
await c.message.answer("Сколько детей?", reply_markup=KB_GLOBAL)
await c.message.answer("Выберите:", reply_markup=kids_inline("zero_kid

```

```
@router.message(ZeroFlow.adults_manual)
```

```
async def zero_adults_manual(m: Message, state: FSMContext):
```

```

    txt = safe_text(m.text)
    if not txt.isdigit():
        await m.answer("Нужно число 1–8. Введите ещё раз:", reply_markup=K
        return
    n = int(txt)
    if not (1 <= n <= 8):
        await m.answer("Взрослые строго 1–8. Введите ещё раз:", reply_mark
        return
    await state.update_data(adults=n)
    await push_state(state, ZeroFlow.kids)
    await m.answer("Сколько детей?", reply_markup=KB_GLOBAL)
    await m.answer("Выберите:", reply_markup=kids_inline("zero_kids").as_m

```

```
@router.callback_query(F.data.startswith("zero_kids:"))
```

```
async def zero_kids_cb(c: CallbackQuery, state: FSMContext):
```

```

    await c.answer()
    val = c.data.split(":", 1)[1]
    if val == "manual":
        await push_state(state, ZeroFlow.kids_manual)
        await c.message.answer("Введите число детей (0–5):", reply_markup=
        return
    k = int(val)
    if not (0 <= k <= 5):
        await c.message.answer("Дети строго 0–5. Выберите снова.", reply_m
        await c.message.answer("Выберите:", reply_markup=kids_inline("zero
        return
    await state.update_data(kids=k)
    if k > 0:
        await push_state(state, ZeroFlow.kids_ages)
        await c.message.answer(f"Укажите возраст каждого ребёнка через зап
    else:
        await push_state(state, ZeroFlow.stars)
        await c.message.answer("Звёзды отеля:", reply_markup=KB_GLOBAL)
        await c.message.answer("Выберите:", reply_markup=stars_inline("zer

```

```
@router.message(ZeroFlow.kids_manual)
```

```

async def zero_kids_manual(m: Message, state: FSMContext):
    txt = safe_text(m.text)
    if not txt.isdigit():
        await m.answer("Нужно число 0-5. Введите ещё раз:", reply_markup=K
        return
    k = int(txt)
    if not (0 <= k <= 5):
        await m.answer("Дети строго 0-5. Введите ещё раз:", reply_markup=K
        return
    await state.update_data(kids=k)
    if k > 0:
        await push_state(state, ZeroFlow.kids_ages)
        await m.answer(f"Укажите возраст каждого ребёнка через запятую (вс
    else:
        await push_state(state, ZeroFlow.stars)
        await m.answer("Звёзды отеля:", reply_markup=KB_GLOBAL)
        await m.answer("Выберите:", reply_markup=stars_inline("zero_stars"

@router.message(ZeroFlow.kids_ages)
async def zero_kids_ages(m: Message, state: FSMContext):
    data = await state.get_data()
    k = int(data.get("kids", 0))
    parts = [p.strip() for p in (m.text or "").split(",") if p.strip()]
    if len(parts) != k:
        await m.answer(f"Нужно указать ровно {k} возраст(ов). Попробуйте е
        return
    ages = []
    for p in parts:
        if not re.fullmatch(r"\d{1,2}", p):
            await m.answer("Возраст должен быть числом. Введите ещё раз:",
            return
        ages.append(int(p))
    await state.update_data(kids_ages=ages)
    await push_state(state, ZeroFlow.stars)
    await m.answer("Звёзды отеля:", reply_markup=KB_GLOBAL)
    await m.answer("Выберите:", reply_markup=stars_inline("zero_stars").as

@router.callback_query(F.data.startswith("zero_stars:"))
async def zero_stars_cb(c: CallbackQuery, state: FSMContext):
    await c.answer()
    val = c.data.split(":", 1)[1]
    if val == "manual":
        await push_state(state, ZeroFlow.stars_manual)

```

```
        await c.message.answer("Введите звёзды отеля текстом (например: 4★)
        return
    await state.update_data(stars=val)
    await push_state(state, ZeroFlow.dates_mode)
    await c.message.answer("Даты поездки:", reply_markup=KB_GLOBAL)
    await c.message.answer("Выберите:", reply_markup=dates_mode_inline("ze

@router.message(ZeroFlow.stars_manual)
async def zero_stars_manual(m: Message, state: FSMContext):
    txt = safe_text(m.text)
    if not txt:
        await m.answer("Введите звёзды отеля:", reply_markup=KB_GLOBAL)
        return
    await state.update_data(stars=txt)
    await push_state(state, ZeroFlow.dates_mode)
    await m.answer("Даты поездки:", reply_markup=KB_GLOBAL)
    await m.answer("Выберите:", reply_markup=dates_mode_inline("zero_dates

@router.callback_query(F.data.startswith("zero_dates:"))
async def zero_dates_cb(c: CallbackQuery, state: FSMContext):
    await c.answer()
    mode = c.data.split(":", 1)[1]
    if mode == "manual":
        await push_state(state, ZeroFlow.dates_manual)
        await c.message.answer("Введите даты текстом (например: 10.01–20.0
        return
    if mode == "exact":
        await push_state(state, ZeroFlow.dates_manual)
        await c.message.answer("Введите точные даты (например: 10.01–20.01
        return
    if mode == "approx":
        await c.message.answer("Выберите примерный период:", reply_markup=
        await c.message.answer("Период:", reply_markup=dates_approx_inline
        return

@router.callback_query(F.data.startswith("zero_dates_approx:"))
async def zero_dates_approx_cb(c: CallbackQuery, state: FSMContext):
    await c.answer()
    val = c.data.split(":", 1)[1]
    if val == "manual":
        await push_state(state, ZeroFlow.dates_manual)
        await c.message.answer("Введите период текстом (например: 'июль 20
        return
```



```
await state.update_data(dates=val)
await push_state(state, ZeroFlow.budget)
await c.message.answer("Бюджет:", reply_markup=KB_GLOBAL)
await c.message.answer("Выберите:", reply_markup=budget_inline("zero_b
```

```
@router.message(ZeroFlow.dates_manual)
```

```
async def zero_dates_manual(m: Message, state: FSMContext):
```

```
    txt = safe_text(m.text)
```

```
    if not txt:
```

```
        await m.answer("Введите даты/период:", reply_markup=KB_GLOBAL)
```

```
        return
```

```
    await state.update_data(dates=txt)
```

```
    await push_state(state, ZeroFlow.budget)
```

```
    await m.answer("Бюджет:", reply_markup=KB_GLOBAL)
```

```
    await m.answer("Выберите:", reply_markup=budget_inline("zero_budget").
```

```
@router.callback_query(F.data.startswith("zero_budget:"))
```

```
async def zero_budget_cb(c: CallbackQuery, state: FSMContext):
```

```
    await c.answer()
```

```
    val = c.data.split(":", 1)[1]
```

```
    if val == "manual":
```

```
        await push_state(state, ZeroFlow.budget_manual)
```

```
        await c.message.answer("Введите бюджет текстом (например: 150 000
```

```
        return
```

```
    await state.update_data(budget=val)
```

```
    await push_state(state, ZeroFlow.rest_type)
```

```
    await c.message.answer("Тип отдыха:", reply_markup=KB_GLOBAL)
```

```
    await c.message.answer("Выберите:", reply_markup=rest_type_inline("zer
```

```
@router.message(ZeroFlow.budget_manual)
```

```
async def zero_budget_manual(m: Message, state: FSMContext):
```

```
    txt = safe_text(m.text)
```

```
    if not txt:
```

```
        await m.answer("Введите бюджет текстом:", reply_markup=KB_GLOBAL)
```

```
        return
```

```
    await state.update_data(budget=txt)
```

```
    await push_state(state, ZeroFlow.rest_type)
```

```
    await m.answer("Тип отдыха:", reply_markup=KB_GLOBAL)
```

```
    await m.answer("Выберите:", reply_markup=rest_type_inline("zero_type"))
```

```
@router.callback_query(F.data.startswith("zero_type:"))
```

```
async def zero_type_cb(c: CallbackQuery, state: FSMContext):
```

```
    await c.answer()
```

```

val = c.data.split(":", 1)[1]
if val == "manual":
    await push_state(state, ZeroFlow.rest_type_manual)
    await c.message.answer("Введите тип отдыха текстом:", reply_markup=
    return

await state.update_data(rest_type=val)

# load directions from sheet
try:
    dirs = get_available_directions()
except Exception as e:
    await c.message.answer(f"Ошибка чтения Google Sheets: {e}", reply_
    return

if not dirs:
    await c.message.answer("В базе пока нет направлений.", reply_marku
    return

await state.update_data(dir_page=0)
await push_state(state, ZeroFlow.choose_direction)
await c.message.answer("Выберите направление из доступных в базе:", re
await c.message.answer("Направления:", reply_markup=paginate_buttons(d

@router.message(ZeroFlow.rest_type_manual)
async def zero_type_manual(m: Message, state: FSMContext):
    txt = safe_text(m.text)
    if not txt:
        await m.answer("Введите тип отдыха текстом:", reply_markup=KB_GLOB
        return
    await state.update_data(rest_type=txt)

    try:
        dirs = get_available_directions()
    except Exception as e:
        await m.answer(f"Ошибка чтения Google Sheets: {e}", reply_markup=K
        return

    if not dirs:
        await m.answer("В базе пока нет направлений.", reply_markup=KB_GLO
        return

    await state.update_data(dir_page=0)

```

```
await push_state(state, ZeroFlow.choose_direction)
await m.answer("Выберите направление из доступных в базе:", reply_mark
await m.answer("Направления:", reply_markup=paginate_buttons(dirs, 0,

@router.callback_query(F.data.startswith("zero_dir:page:"))
async def zero_dir_page(c: CallbackQuery, state: FSMContext):
    await c.answer()
    page = int(c.data.split(":")[-1])
    await state.update_data(dir_page=page)
    dirs = get_available_directions()
    await c.message.edit_reply_markup(reply_markup=paginate_buttons(dirs,

@router.callback_query(F.data.startswith("zero_dir:pick:"))
async def zero_dir_pick(c: CallbackQuery, state: FSMContext, bot: Bot):
    await c.answer()
    direction = c.data.split(":", 2)[2]
    await state.update_data(direction=direction)

    tours = get_tours_for_direction(direction)
    if not tours:
        await c.message.answer("По этому направлению в базе нет туров. Выб
        return

    profile = build_zero_profile_text(await state.get_data())
    ranked = rank_tours(profile, tours)
    order = [i for i, _ in ranked] if ranked else list(range(len(tours)))

    await state.update_data(tours=tours, tour_order=order, tour_offset=0,
    await push_state(state, ZeroFlow.show_tours)
    await show_next_tours(bot, c.message.chat.id, state)

async def show_next_tours(bot: Bot, chat_id: int, state: FSMContext):
    data = await state.get_data()
    tours = data.get("tours", [])
    order = data.get("tour_order", [])
    offset = int(data.get("tour_offset", 0))

    if not tours or not order:
        await bot.send_message(chat_id, "Туры не найдены. Попробуйте выбра
        return

    chunk = order[offset:offset+2]
    if not chunk:
```

```

    await bot.send_message(chat_id, "Больше вариантов нет. Подтвердите")
    await push_state(state, ZeroFlow.summary)
    await render_zero_summary_to_chat(bot, chat_id, state)
    return

for pos, tour_idx in enumerate(chunk, start=1):
    t = tours[tour_idx]
    url = t.get("URL", "")
    desc = t.get("описание типа", "") or "Описание скоро будет добавле
n_global = offset + pos

caption = f"Вариант типа #{n_global}\n\n{desc}"
kb = InlineKeyboardBuilder()
kb.row(InlineKeyboardButton(text="Выбрать этот тип", callback_data

if url:
    try:
        await bot.send_photo(chat_id, photo=url, caption=caption,
    except Exception:
        await bot.send_message(chat_id, caption + f"\n\nФото: {url
else:
    await bot.send_message(chat_id, caption, reply_markup=kb.as_ma

new_offset = offset + len(chunk)
await state.update_data(tour_offset=new_offset)

if new_offset < len(order):
    kb_more = InlineKeyboardBuilder()
    kb_more.row(InlineKeyboardButton(text="Показать ещё варианты", cal
    await bot.send_message(chat_id, "Показать ещё варианты?", reply_ma
else:
    await bot.send_message(chat_id, "Если хотите – выберите один из ва

@router.callback_query(F.data == "zero_more")
async def zero_more(c: CallbackQuery, state: FSMContext, bot: Bot):
    await c.answer()
    await show_next_tours(bot, c.message.chat.id, state)

@router.callback_query(F.data.startswith("zero_tour_pick:"))
async def zero_pick_tour(c: CallbackQuery, state: FSMContext, bot: Bot):
    await c.answer()
    tour_idx = int(c.data.split(":")[1])
    data = await state.get_data()

```

```

tours = data.get("tours", [])
if not tours or tour_idx >= len(tours):
    await c.message.answer("Не получилось выбрать тур. Попробуйте снова")
    return
await state.update_data(chosen_tour=tours[tour_idx])
await push_state(state, ZeroFlow.summary)
await render_zero_summary_to_chat(bot, c.message.chat.id, state)

async def render_zero_summary_to_chat(bot: Bot, chat_id: int, state: FSMContext):
    data = await state.get_data()
    txt = [
        "📌 Анкета:",
        f"Направление: {data.get('direction')}",
        f"Состав: взрослые {data.get('adults')}, дети {data.get('kids')}"
        f"Звёзды отеля: {data.get('stars')}",
        f"Даты: {data.get('dates')}",
        f"Бюджет: {data.get('budget')}",
        f"Тип отдыха: {data.get('rest_type')}",
        "",
    ]
    chosen = data.get("chosen_tour")
    if chosen:
        txt += [
            "👁️ Подобранный тур:",
            f"Описание: {chosen.get('описание тура', '') or '-'}",
            f"Картинка: {chosen.get('URL', '') or '-'}",
            "",
        ]
    txt.append("Подтвердить анкету или изменить?")

    kb = InlineKeyboardBuilder()
    kb.row(InlineKeyboardButton(text="✅ Подтвердить анкету", callback_data="confirm"))
    kb.row(InlineKeyboardButton(text="✏️ Изменить", callback_data="zero_edit"))
    await bot.send_message(chat_id, "\n".join(txt), reply_markup=kb.as_markup())

@router.callback_query(F.data == "zero_edit")
async def zero_edit(c: CallbackQuery, state: FSMContext):
    await c.answer()
    await push_state(state, ZeroFlow.edit_field)
    kb = InlineKeyboardBuilder()
    kb.row(InlineKeyboardButton(text="Направление", callback_data="zero_edit_direction"))
    kb.row(InlineKeyboardButton(text="Состав", callback_data="zero_edit_composition"))
    kb.row(InlineKeyboardButton(text="Звёзды отеля", callback_data="zero_edit_stars"))

```

```
kb.row(InlineKeyboardButton(text="Даты", callback_data="zero_edit_fiel
kb.row(InlineKeyboardButton(text="Бюджет", callback_data="zero_edit_fi
kb.row(InlineKeyboardButton(text="Тип отдыха", callback_data="zero_edi
kb.row(InlineKeyboardButton(text="Тип", callback_data="zero_edit_field
await c.message.answer("Что изменить?", reply_markup=kb.as_markup())
```

```
@router.callback_query(F.data.startswith("zero_edit_field:"))
```

```
async def zero_edit_field_pick(c: CallbackQuery, state: FSMContext, bot: B
    await c.answer()
    field = c.data.split(":")[1]
```

```
if field == "direction":
    await state.update_data(dir_page=0, chosen_tour=None, tour_offset=
    await state.set_state(ZeroFlow.choose_direction)
    dirs = get_available_directions()
    await c.message.answer("Выберите направление из базы:", reply_mark
    await c.message.answer("Направления:", reply_markup=paginate_butto
    return
```

```
if field == "party":
    await state.set_state(ZeroFlow.adults)
    await c.message.answer("Сколько взрослых?", reply_markup=KB_GLOBAL
    await c.message.answer("Выберите:", reply_markup=adults_inline("ze
    return
```

```
if field == "stars":
    await state.set_state(ZeroFlow.stars)
    await c.message.answer("Звёзды отеля:", reply_markup=KB_GLOBAL)
    await c.message.answer("Выберите:", reply_markup=stars_inline("zer
    return
```

```
if field == "dates":
    await state.set_state(ZeroFlow.dates_mode)
    await c.message.answer("Даты поездки:", reply_markup=KB_GLOBAL)
    await c.message.answer("Выберите:", reply_markup=dates_mode_inline
    return
```

```
if field == "budget":
    await state.set_state(ZeroFlow.budget)
    await c.message.answer("Бюджет:", reply_markup=KB_GLOBAL)
    await c.message.answer("Выберите:", reply_markup=budget_inline("ze
    return
```

```
if field == "rest_type":
    await state.set_state(ZeroFlow.rest_type)
    await c.message.answer("Тип отдыха:", reply_markup=KB_GLOBAL)
    await c.message.answer("Выберите:", reply_markup=rest_type_inline)
    return

if field == "tour":
    data = await state.get_data()
    direction = data.get("direction")
    tours = get_tours_for_direction(direction)
    profile = build_zero_profile_text(data)
    ranked = rank_tours(profile, tours)
    order = [i for i, _ in ranked] if ranked else list(range(len(tours)))
    await state.update_data(tours=tours, tour_order=order, tour_offset=0)
    await state.set_state(ZeroFlow.show_tours)
    await show_next_tours(bot, c.message.chat.id, state)
    return

@router.callback_query(F.data == "zero_confirm")
async def zero_confirm(c: CallbackQuery, state: FSMContext):
    await c.answer()
    await push_state(state, ZeroFlow.contact_phone)
    await c.message.answer("Отправьте номер телефона (кнопкой) или напишите")

@router.message(ZeroFlow.contact_phone)
async def zero_phone(m: Message, state: FSMContext):
    phone = None
    if m.contact and m.contact.phone_number:
        phone = normalize_phone(m.contact.phone_number)
    else:
        phone = normalize_phone(m.text or "")
    if not phone:
        await m.answer("Не распознал телефон. Отправьте кнопкой или напишите")
        return
    await state.update_data(phone=phone)
    await push_state(state, ZeroFlow.contact_name)
    await m.answer("Введите Фамилию Имя:", reply_markup=KB_GLOBAL)

@router.message(ZeroFlow.contact_name)
async def zero_name(m: Message, state: FSMContext):
    fio = safe_text(m.text)
    if len(fio.split()) < 2:
        await m.answer("Нужно в формате «Фамилия Имя». Введите ещё раз:",
```

```

        return
    data = await state.get_data()
    phone = data.get("phone", "")
    await m.answer(
        f"Спасибо, {fio}! Мы отправили вашу заявку менеджеру – скоро с вам
        reply_markup=KB_START
    )
    await reset_to_start(state)

# -----
# 16) TTS callbacks (idea branch messages)
# -----
@router.callback_query(F.data.startswith("tts:"))
async def tts_cb(c: CallbackQuery, bot: Bot):
    await c.answer()
    key = c.data.split(":", 1)[1]
    payload = TTS_STORE.get(key)
    if not payload:
        await c.message.answer("Не нашёл текст для озвучки (сообщение слиш
        return
    text = payload.get("text", "")
    try:
        mp3 = await openai_tts_mp3_bytes(text)
        from aiogram.types import BufferedInputFile
        audio = BufferedInputFile(mp3, filename="tts.mp3")
        await bot.send_audio(c.message.chat.id, audio=audio, caption="🔊 0
    except Exception as e:
        await c.message.answer(f"Ошибка озвучки: {e}")

# -----
# 17) Fallback
# -----
@router.message()
async def fallback(m: Message, state: FSMContext):
    st = await state.get_state()
    if not st:
        await m.answer("Нажмите /start", reply_markup=KB_START)
        return
    await m.answer("Пожалуйста, используйте кнопки или ответьте на вопрос.

# -----
# 18) Run bot
# -----

```



```
async def main():
    bot = Bot(TELEGRAM_BOT_TOKEN)
    dp = Dispatcher(storage=MemoryStorage())
    dp.include_router(router)

    try:
        from aiogram.types import BotCommand
        await bot.set_my_commands([
            BotCommand(command="start", description="Начать заново"),
            BotCommand(command="help", description="Помощь"),
            BotCommand(command="contacts", description="Контакты менеджера"),
            BotCommand(command="site", description="Сайт"),
        ])
    except Exception:
        pass

    try:
        await bot.delete_webhook(drop_pending_updates=True)
    except Exception:
        pass

    await dp.start_polling(bot, allowed_updates=dp.resolve_used_update_types())

asyncio.run(main())
```

